

User Manual

for S32 FR Driver

Document Number: UM11FRASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	7
3.5 Driver Limitations	8
3.6 Driver usage and configuration tips	8
3.7 Runtime errors	8
3.8 Symbolic Names Disclaimer	10
4 Tresos Configuration Plug-in	11
4.1 Module Fr	13
4.2 Container CommonPublishedInformation	13
4.3 Parameter ArReleaseMajorVersion	15
4.4 Parameter ArReleaseMinorVersion	15
4.5 Parameter ArReleaseRevisionVersion	16
4.6 Parameter ModuleId	16
4.7 Parameter SwMajorVersion	17
4.8 Parameter SwMinorVersion	17
4.9 Parameter SwPatchVersion	18
4.10 Parameter VendorApiInfix	18
4.11 Parameter VendorId	19
4.12 Container FrGeneral	19
4.13 Parameter FrPrepareLPduSupport	19
4.14 Parameter FrReconfigLPduSupport	20
4.15 Parameter FrDisableLPduSupport	20
4.16 Parameter FrExtendedLPduReporting	21
4.17 Parameter FrCtrlTestCount	21
4.18 Parameter FrDevErrorDetect	22
4.19 Parameter FrIndex	22
4.20 Parameter FrNumCtrlSupported	23
4.21 Parameter FrRxStringentCheck	23

4.22 Parameter FrRxStringentLengthCheck	24
4.23 Parameter FrVersionInfoApi	24
4.24 Reference FrEcucPartitionRef	25
4.25 Container VendorSpecific	25
4.26 Parameter FrUnusedBitValue	26
4.27 Parameter ConnectedCC	26
4.28 Parameter FrDisableDemReportErrorStatus	27
4.29 Parameter FrEnableUserModeSupport	27
4.30 Parameter FrEnableMulticoreSupport	28
4.31 Parameter FrTimeoutMethod	28
4.32 Parameter FrTimeoutDuration	29
4.33 Container FrMultipleConfiguration	29
4.34 Container FrController	29
4.35 Parameter FrCtrlIdx	30
4.36 Parameter FrPAllowHaltDueToClock	30
4.37 Parameter FrPAllowPassiveToActive	31
4.38 Parameter FrPChannels	31
4.39 Parameter FrPClusterDriftDamping	32
4.40 Parameter FrPDecodingCorrection	32
4.41 Parameter FrPDelayCompensationA	33
4.42 Parameter FrPDelayCompensationB	33
4.43 Parameter FrPExternalSync	34
4.44 Parameter FrPFallBackInternal	34
4.45 Parameter FrPKeySlotId	35
4.46 Parameter FrPKeySlotOnlyEnabled	35
4.47 Parameter FrPKeySlotUsedForStartup	36
4.48 Parameter FrPKeySlotUsedForSync	36
4.49 Parameter FrPLatestTx	37
4.50 Parameter FrPMacroInitialOffsetA	37
4.51 Parameter FrPMacroInitialOffsetB	38
4.52 Parameter FrPMicroInitialOffsetA	38
4.53 Parameter FrPMicroInitialOffsetB	38
4.54 Parameter FrPMicroPerCycle	39
4.55 Parameter FrPNmVectorEarlyUpdate	39
4.56 Parameter FrPOffsetCorrectionOut	40
4.57 Parameter FrPOffsetCorrectionStart	40
4.58 Parameter FrPPayloadLengthDynMax	41
4.59 Parameter FrPRateCorrectionOut	41
4.60 Parameter FrPSamplesPerMicrotick	42
4.61 Parameter FrPSecondKeySlotId	42

4.62	Parameter	FrPTwoKeySlotMode	43
4.63	Parameter	FrPWakeupChannel	43
4.64	Parameter	FrPWakeupPattern	44
4.65	Parameter	FrPdAcceptedStartupRange	44
4.66	Parameter	FrPdListenTimeout	45
4.67	Parameter	FrPdMicrotick	45
4.68	Reference	FrCtrlEcucPartitionRef	46
4.69	Container	VendorSpecific	46
4.70	Parameter	CCBaseAddress	47
4.71	Parameter	SystemFrequency	47
4.72	Parameter	SystemMemoryAccessTimeOut	48
4.73	Parameter	ChannelBitrate	48
4.74	Parameter	SingleChannelModeEnabled	49
4.75	Parameter	ForceReconfiguration	49
4.76	Container	FrTestModeEnable	50
4.77	Parameter	FrTestModeNumberOfResources	51
4.78	Container	FrAbsoluteTimer	51
4.79	Parameter	FrAbsTimerIdx	51
4.80	Container	FrControllerDemEventParameterRefs	52
4.81	Reference	FR_E_CTRL_TESTRESULT	52
4.82	Container	FrFifo	53
4.83	Parameter	FrAdmitWithoutMessageId	53
4.84	Parameter	FrBaseCycle	54
4.85	Parameter	FrChannels	54
4.86	Parameter	FrCycleRepetition	55
4.87	Parameter	FrFifoDepth	55
4.88	Parameter	FrMsgIdMask	56
4.89	Parameter	FrMsgIdMatch	56
4.90	Container	FrRange	57
4.91	Parameter	FrRangeMin	57
4.92	Parameter	FrRangeMax	58
4.93	Container	FrVendorSpecific	58
4.94	Parameter	FrRangeMode	58
5	Module Index		60
5.1	Software Specification		60
6	Module Documentation		61
6.1	Flexray		61
6.1.1	Detailed Description		61
6.1.2	Data Structure Documentation		65

6.1.3 Types Reference	84
6.1.4 Enum Reference	85
6.1.5 Function Reference	93
6.2 FR_DRIVER	112
6.2.1 Detailed Description	112
6.2.2 Macro Definition Documentation	114
6.2.3 Function Reference	117



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR Flexray driver for S32 platform. AUTOSAR Flexray driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR Flexray driver requirements and APIs are described in the AUTOSAR Flexray driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Fr Driver	AUTOSAR Release 4.4.0
2	Reference Manual	S32G2 Reference Manual, Rev. 7, February 2023
		S32G3 Reference Manual, Rev.3, Feb 2023
		S32R45 Reference Manual, Rev. 3, 12/2021
3	Datasheet	S32G2 Data Sheet, Rev 6, Dec 2022
		S32G3 Data Sheet, Rev 2, RC, Feb 2023
		VR5510 Data Sheet, Rev 5, April 2022
		S32R45 Data Sheet, Rev. 3, RC — 11/2022
4	Errata	S32G2: Mask Set Errata for Mask 0P77B v3.0
		S32G2: Mask Set Errata for Mask 1P77B v1.0
		S32G3: Mask Set Errata for Mask 1P72B v2.1
		S32R45: Mask Set Errata for Mask P57D, Rev. 2.0

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.4 Rev0000 Flexray Driver Software Specification document (See Table [Reference List](#)).

3.2 Driver Design Summary

The driver provides services for the following (FlexRay controller) features:

- Stores configuration data address to enable subsequent API calls to access the configuration data.
- Configures all FlexRay cluster and node configuration parameters.
- Provides set of API functions which allow to start communication or stop communication (immediately or at the end of the current FlexRay communication cycle).
- Data transmission and reception in both static and dynamic parts of communication cycle is serviced by several Fr module API functions.
- Protocol state machine control of the FlexRay CC is implemented by several Fr module API functions.

- Provides information about FlexRay network global time.
- Evaluates whether the CC is synchronized to the global FlexRay time.
- API functions for configuration of wakeup channel and sending of the wakeup pattern are implemented.
- Individual message buffer reconfiguration is supported.
- Support of two independent receive FIFOs (one receive FIFO per channel).
- Driver provides set of API functions to support absolute timer.
- Driver provides set of API functions for timer interrupt servicing.
- Driver provides API for obtaining the node and cluster configuration parameters from the FlexRay CC Protocol Configuration Registers.
- Driver provides DEM and DET error notification.

3.3 Hardware Resources

The hardware configured by the Fr driver is the same between derivatives.

- Reference Manual naming: FR_0

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR Flexray Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the Flexray Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the Flexray driver.

Requirement	Status	Description	Notes
SWS_Fr_00116	N/S	The code file structure shall not be completely defined within this specification.	Not related to FR driver
SWS_Fr_00479	N/S	The rationale of SWS_Fr_00097 is to allow object code module integration.	Not requirement, just information

3.5 Driver Limitations

The Flexray Driver has the following limitations:

- Interrupt and callback functions are only supported for IPL driver. If the ASR's user wants to use interrupt, they must be used by FRIF driver.
- FIFOA and FIFOB interrupt flags will not be immediately cleared upon entering the interrupt handler function due to hardware. So IPL's user if use interrupt need call Flexray_Ip_ReceiveFIFO in fifo callback function to receive and clear the flags.

3.6 Driver usage and configuration tips

The FlexRay drivers have some features which are not (fully) described by AUTOSAR SWS (i.e NON-ASR features). This chapter is show-casing how to configure/enable those advanced features. When using the FlexRay driver, is recommended to check the default values option first. FlexRay Ip provides presets to switch between desired configuration speeds.

3.7 Runtime errors

The driver generates the following DEM errors at runtime.

Function	Error Code	Condition triggering the error
Fr_ControllerInit()	FR_E_CTRL_TESTRESULT	CC is not accessible/ CC can not be disabled/ CC can not enter POC:Config state/ There are errors when driver configures the transmit, receive message buffers, shadow buffers and FIFO arrays/ CC can not leave POC: Config state/ There are errors when reading back or incorrect value when comparing cluster and configuration values to reference values held in the configuration/ Some interrupt flag was not clear or some interrupt was not disabled.
Fr_StartCommunication()	FR_E_CTRL_TESTRESULT	CC is not accessible/ CHI command was not accepted by PE due to BSY flag.
Fr_AllowColdstart()	FR_E_CTRL_TESTRESULT	CC is not accessible/ CHI command was not accepted by PE due to BSY flag.
Fr_HaltCommunication()	FR_E_CTRL_TESTRESULT	CC is not accessible/ CHI command was not accepted by PE due to BSY flag.
Fr_AbortCommunication()	FR_E_CTRL_TESTRESULT	CC is not accessible/ CHI command was not accepted by PE due to BSY flag.
Fr_SendWUP()	FR_E_CTRL_TESTRESULT	CC is not accessible/ CHI command was not accepted by PE due to BSY flag.

Function	Error Code	Condition triggering the error
Fr_SetWakeupChannel()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Error occurred during set wakeup channel.
Fr_GetPOCStatus()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Can get POC status due to hardware errors.
Fr_GetGlobalTime()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Incorrect cycle and macrotick value reading from registers.
Fr_SetAbsoluteTimer()	FR_E_CTRL_TESTRESULT	CC is not accessible.
Fr_CancelAbsoluteTimer()	FR_E_CTRL_TESTRESULT	CC is not accessible.
Fr_EnableAbsoluteTimerIRQ()	FR_E_CTRL_TESTRESULT	CC is not accessible.
Fr_AckAbsoluteTimerIRQ()	FR_E_CTRL_TESTRESULT	CC is not accessible.
Fr_DisableAbsoluteTimerIRQ()	FR_E_CTRL_TESTRESULT	CC is not accessible.
Fr_GetAbsoluteTimerIRQStatus()	FR_E_CTRL_TESTRESULT	CC is not accessible.
Fr_GetNmVector()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Hardware errors may occur.
Fr_GetChannelStatus()	FR_E_CTRL_TESTRESULT	CC is not accessible.
Fr_GetClockCorrection()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Incorrect rate and offset correction values from FR_RTC←ORVR and FR_OFCORVR registers.
Fr_GetSyncFrameList()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Hardware errors may occur.
Fr_GetWakeupRxStatus()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Can not clear flag in register FR_PSR3.
Fr_TransmitTxLPdu()	FR_E_CTRL_TESTRESULT	CC is not accessible/ MB is not configured for Tx/ The MB is not locked.
Fr_ReceiveRxLPdu()	FR_E_CTRL_TESTRESULT	CC is not accessible/ MB is not configured for Rx.
Fr_ReceiveRxLPdu()	FR_E_LPDU_SLOTSTATUS	Single slot status error bit is set.
Fr_CheckTxLPduStatus()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Data was not ready for transmission.
Fr_CheckTxLPduStatus()	FR_E_LPDU_SLOTSTATUS	CC is not accessible/ Single slot status error bit is set.
Fr_PrepareLPdu()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Hardware error, can not enable, disable Message Buffer or invalid MB configuration.
Fr_CancelTxLPdu()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Data not ready for transmission/ Can not lock or unlock the MB.
Fr_ReconfigLPdu()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Data not ready for transmission/ Can not enable or disable MB.
Fr_DisableLPdu()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Can not disable the MB.
Fr_ReadCCConfig()	FR_E_CTRL_TESTRESULT	CC is not accessible/ Incorrect rate and offset correction values from FR_RTC←ORVR and FR_OFCORVR registers.
Fr_GetNumOfStartupFrames()	FR_E_CTRL_TESTRESULT	CC is not accessible.
Fr_DeInit()	FR_E_CTRL_TESTRESULT	Hardware error or incorrect sequence.

Function	Error Code	Condition triggering the error
----------	------------	--------------------------------

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Fr](#)
 - Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)
 - Container [FrGeneral](#)
 - * Parameter [FrPrepareLPduSupport](#)
 - * Parameter [FrReconfigLPduSupport](#)
 - * Parameter [FrDisableLPduSupport](#)
 - * Parameter [FrExtendedLPduReporting](#)
 - * Parameter [FrCtrlTestCount](#)
 - * Parameter [FrDevErrorDetect](#)
 - * Parameter [FrIndex](#)
 - * Parameter [FrNumCtrlSupported](#)
 - * Parameter [FrRxStringentCheck](#)
 - * Parameter [FrRxStringentLengthCheck](#)
 - * Parameter [FrVersionInfoApi](#)
 - * Reference [FrEcucPartitionRef](#)
 - * Container [VendorSpecific](#)
 - Parameter [FrUnusedBitValue](#)
 - Parameter [ConnectedCC](#)
 - Parameter [FrDisableDemReportErrorStatus](#)
 - Parameter [FrEnableUserModeSupport](#)
 - Parameter [FrEnableMulticoreSupport](#)

- Parameter [FrTimeoutMethod](#)
- Parameter [FrTimeoutDuration](#)
- Container [FrMultipleConfiguration](#)
 - * Container [FrController](#)
 - Parameter [FrCtrlIdx](#)
 - Parameter [FrPAllowHaltDueToClock](#)
 - Parameter [FrPAllowPassiveToActive](#)
 - Parameter [FrPChannels](#)
 - Parameter [FrPClusterDriftDamping](#)
 - Parameter [FrPDecodingCorrection](#)
 - Parameter [FrPDelayCompensationA](#)
 - Parameter [FrPDelayCompensationB](#)
 - Parameter [FrPExternalSync](#)
 - Parameter [FrPFallBackInternal](#)
 - Parameter [FrPKeySlotId](#)
 - Parameter [FrPKeySlotOnlyEnabled](#)
 - Parameter [FrPKeySlotUsedForStartup](#)
 - Parameter [FrPKeySlotUsedForSync](#)
 - Parameter [FrPLatestTx](#)
 - Parameter [FrPMacroInitialOffsetA](#)
 - Parameter [FrPMacroInitialOffsetB](#)
 - Parameter [FrPMicroInitialOffsetA](#)
 - Parameter [FrPMicroInitialOffsetB](#)
 - Parameter [FrPMicroPerCycle](#)
 - Parameter [FrPNmVectorEarlyUpdate](#)
 - Parameter [FrPOffsetCorrectionOut](#)
 - Parameter [FrPOffsetCorrectionStart](#)
 - Parameter [FrPPayloadLengthDynMax](#)
 - Parameter [FrPRateCorrectionOut](#)
 - Parameter [FrPSamplesPerMicrotick](#)
 - Parameter [FrPSecondKeySlotId](#)
 - Parameter [FrPTwoKeySlotMode](#)
 - Parameter [FrPWakeupChannel](#)
 - Parameter [FrPWakeupPattern](#)
 - Parameter [FrPdAcceptedStartupRange](#)
 - Parameter [FrPdListenTimeout](#)
 - Parameter [FrPdMicrotick](#)
 - Reference [FrCtrlEcucPartitionRef](#)
 - Container [VendorSpecific](#)
 - Parameter [CCBaseAddress](#)
 - Parameter [SystemFrequency](#)
 - Parameter [SystemMemoryAccessTimeOut](#)
 - Parameter [ChannelBitrate](#)
 - Parameter [SingleChannelModeEnabled](#)
 - Parameter [ForceReconfiguration](#)
 - Container [FrTestModeEnable](#)
 - Parameter [FrTestModeNumberOfResources](#)

- Container [FrAbsoluteTimer](#)
- Parameter [FrAbsTimerIdx](#)
- Container [FrControllerDemEventParameterRefs](#)
- Reference [FR_E_CTRL_TESTRESULT](#)
- Container [FrFifo](#)
- Parameter [FrAdmitWithoutMessageId](#)
- Parameter [FrBaseCycle](#)
- Parameter [FrChannels](#)
- Parameter [FrCycleRepetition](#)
- Parameter [FrFifoDepth](#)
- Parameter [FrMsgIdMask](#)
- Parameter [FrMsgIdMatch](#)
- Container [FrRange](#)
- Parameter [FrRangeMin](#)
- Parameter [FrRangeMax](#)
- Container [FrVendorSpecific](#)
- Parameter [FrRangeMode](#)

4.1 Module Fr

Configuration of the Fr(FlexRay driver) module.

Included containers:

- [CommonPublishedInformation](#)
- [FrGeneral](#)
- [FrMultipleConfiguration](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

Tresos Configuration Plug-in

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.4 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.5 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.6 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	81

Property	Value
max	81
min	81

4.7 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.8 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.9 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.10 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

[ModuleName]_[VendorId]_[VendorApiInfix][Api name from SWS].

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.11 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.12 Container FrGeneral

General configuration (parameters) of the FlexRay Driver module.

Included subcontainers:

- [VendorSpecific](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.13 Parameter FrPrepareLPduSupport

Enables or disables API function Fr_PrepareLPdu.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.14 Parameter FrReconfigLPduSupport

Enables or disabled API function Fr_ReconfigLPdu.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.15 Parameter FrDisableLPduSupport

Enables or disabled API function Fr_DisableLPdu.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.16 Parameter FrExtendedLPduReporting

Enables or disables reporting of actual cycle and slot ID by TransmitTxLPdu, Fr_ReceiveRxLPdu, and Fr_CheckTxLPduStatus.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.17 Parameter FrCtrlTestCount

Maximum number of iterations the FlexRay controller hardware test is performed during controller initialization.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
defaultValue	1
max	255
min	0

4.18 Parameter FrDevErrorDetect

Switches the Development Error Detection and Notification on or off.

true: Development Error Detection and Notification enabled.

false: Development Error Detection and Notification disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.19 Parameter FrIndex

Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
defaultValue	0
max	255
min	0

4.20 Parameter FrNumCtrlSupported

Determines the maximum number of communication controllers that the driver supports.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	256
min	1

4.21 Parameter FrRxStringentCheck

If stringent check is enabled (true), received frames are accepted only if:

the SlotStatus shows a valid frame (vSS!ValidFrame)

there was no single SlotStatus error bit set:

vSS!SyntaxError

vSS!ContentError

vSS!Bviolation

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.22 Parameter FrRxStringentLengthCheck

If stringent length check is enabled (true), received frames are accepted only if the received payload length exactly matches the expected payload

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.23 Parameter FrVersionInfoApi

Enables/disables the existence of the Fr_GetVersionInfo API.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.24 Reference FrEcucPartitionRef

Maps the Flexray driver to zero or multiple ECUC partitions to make the modules API available in this partition. The Flexray driver will operate as an independent instance in each of the partitions.

Tags: atp.Status=draft.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.25 Container VendorSpecific

Freescall FlexRay module specific information.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.26 Parameter FrUnusedBitValue

Vendor specific:

This parameter specifies that the function Fr_TransmitTxLPdu sets the remaining bits in the CC's transmission resource to the configured value given by parameter FrUnusedBitValue.

This feature is reused from AutosaR FlexRay Driver specification v2.1 REV0019.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	1
min	0

4.27 Parameter ConnectedCC

Vendor specific:

The type of used Freescale FlexRay device.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	Fr_S32XX
literals	['Fr_S32XX']

4.28 Parameter FrDisableDemReportErrorStatus

Vendor specific:

Switches the Diagnostic Error Reporting OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.29 Parameter FrEnableUserModeSupport

Vendor specific:

When this parameter is enabled, the FlexRay module will adapt to run from User Mode, with the following measures:

(if applicable) a) configuring REG_PROT for FlexRay controller so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

(if applicable) b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

(if applicable) c) other module specific measures

for more information, please see chapter 5.7 User Mode Support in IM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.30 Parameter FrEnableMulticoreSupport

Vendor specific:

Enable Multicore support for Flexray driver

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.31 Parameter FrTimeoutMethod

FrTimeoutMethod: Configures the timeout method.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If SystemTimer or CustomTimer are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM', 'OSIF_COUNTER_DUMMY']

4.32 Parameter FrTimeoutDuration

The unit of measurement is given in number of microseconds.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1000
max	65535
min	1

4.33 Container FrMultipleConfiguration

Configuration of the individual controllers.

Included subcontainers:

- [FrController](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.34 Container FrController

Configuration of the individual controller.

Included subcontainers:

- [VendorSpecific](#)
- [FrAbsoluteTimer](#)
- [FrControllerDemEventParameterRefs](#)
- [FrFifo](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.35 Parameter FrCtrlIdx

Determines index of CC within Fr.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.36 Parameter FrPAllowHaltDueToClock

Boolean flag that controls the transition to the POC:halt state due to a clock synchronization errors. If set to true, the CC is allowed to transition to POC:halt. If set to false, the CC will not transition to the POC:halt state but will enter or remain in the POC:normal passive state (self healing would still be possible)

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.37 Parameter FrPAllowPassiveToActive

Number of consecutive even/odd cycle pairs that must have valid clock correction terms before the CC will be allowed to transition from the POC:normal passive state to POC:normal active state. If set to zero, the CC is not allowed to transition from POC:normal passive to POC:normal active

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	31
min	0

4.38 Parameter FrPChannels

Channels to which the node is connected.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FR_CHANNEL_AB
literals	['FR_CHANNEL_A', 'FR_CHANNEL_AB', 'FR_CHANNEL_B']

4.39 Parameter FrPClusterDriftDamping

Local cluster drift damping factor used for rate correction [Microticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	20
min	0

4.40 Parameter FrPDecodingCorrection

Value used by the receiver to calculate the difference between primary time reference point and secondary time reference point [Microticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	36
max	143
min	12

4.41 Parameter FrPDelayCompensationA

Value used to compensate for reception delays on the indicated channel. This covers assumed propagation delay up to cPropagationDelayMax for microticks in the range of 0.0125us to 0.05us [Microticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	211
min	0

4.42 Parameter FrPDelayCompensationB

Value used to compensate for reception delays on the indicated channel. This covers assumed propagation delay up to cPropagationDelayMax for microticks in the range of 0.0125us to 0.05us [Microticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	211
min	0

4.43 Parameter FrPExternalSync

Flag indicating whether the node is externally synchronized (operating as time gateway sink in an TT-E cluster) or locally synchronized.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.44 Parameter FrPFallBackInternal

Flag indicating whether a time gateway sink node will switch to local clock operation when synchronization with the time gateway source node is lost (FrPFallBackInternal = true) or will instead go to POC:halt (FrPFallBackInternal = false).

This feature is not supported by the 2.1 controller therefore it shall be always set to false.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.45 Parameter FrPKeySlotId

ID of the key slot, i.e., the slot used to transmit the startup frame, sync frame, or designated key slot frame. If this parameter is set to zero the node does not have a key slot.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	1023
min	0

4.46 Parameter FrPKeySlotOnlyEnabled

Flag indicating whether or not the node shall enter key slot only mode following startup.

Use the 2.1 specification parameter pSingleSlotEnabled value here.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.47 Parameter FrPKeySlotUsedForStartup

Flag indicating whether the key slot is used to transmit a startup frame.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.48 Parameter FrPKeySlotUsedForSync

Flag indicating whether the key slot is used to transmit a sync frame.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.49 Parameter FrPLatestTx

Number of the last minislot in which a frame transmission can start in the dynamic segment.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	157
max	7988
min	0

4.50 Parameter FrPMacroInitialOffsetA

Integer number of macroticks between the static slot boundary and the following macrotick boundary of the secondary time reference point based on the nominal macrotick duration [Macroticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8
max	68
min	2

4.51 Parameter FrPMacroInitialOffsetB

Integer number of macroticks between the static slot boundary and the following macrotick boundary of the secondary time reference point based on the nominal macrotick duration [Macroticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8
max	68
min	2

4.52 Parameter FrPMicroInitialOffsetA

Number of microticks between the secondary time reference point

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	24
max	239
min	0

4.53 Parameter FrPMicroInitialOffsetB

Number of microticks between the secondary time reference point

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	24
max	239
min	0

4.54 Parameter FrPMicroPerCycle

Nominal number of microticks in the communication cycle of the local node. If nodes have different microtick durations this number will differ from node to node [Microticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	200000
max	1280000
min	640

4.55 Parameter FrPNmVectorEarlyUpdate

Flag indicating when the update of the Network Management Vector

The 2.1 controller supports only update at the end of the communication cycle, therefore the value shall be always false.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.56 Parameter FrPOffsetCorrectionOut

Magnitude of the maximum permissible offset correction value [Microticks]. Remark: Upper limit 15567 for FlexRay Protocol 2.1 Rev A compliance. Remark: Lower limit 15 for FlexRay Protocol 3.0 compliance.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	120
max	16082
min	13

4.57 Parameter FrPOffsetCorrectionStart

Start of the offset correction phase within the NIT, expressed as the number of macroticks from the start of cycle [Macroticks].

Use the value of gOffsetCorrectionStart from the specification 2.1A .

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	4920
max	15999
min	7

4.58 Parameter FrPPayloadLengthDynMax

Maximum payload length for dynamic frames [16 bit words].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	8
max	127
min	0

4.59 Parameter FrPRateCorrectionOut

Magnitude of the maximum permissible rate correction value and the maximum drift offset between two nodes operating with unsynchronized clocks for one communication cycle [Microticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	962
max	3846
min	2

4.60 Parameter FrPSamplesPerMicrotick

Number of samples per microtick. Remark: Allowed range N1SAMPLES, N2SAMPLES for FlexRay Protocol 3.0 compliance.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	N2SAMPLES
literals	['N1SAMPLES', 'N2SAMPLES', 'N4SAMPLES']

4.61 Parameter FrPSecondKeySlotId

ID of the second key slot, in which a second startup frame shall be sent when operating as a coldstart node in a TT-L or TT-D cluster.

If this parameter is set to zero the node does not have a second key slot.

The 2.1 controller does not support this 3.0 specification feature therefore the value shall be always 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	1023
min	0

4.62 Parameter FrPTwoKeySlotMode

Flag indicating whether node operates as a coldstart node in a TT-E or TT-L cluster.

The 2.1 controller does not support this feature therefore the value is always false.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.63 Parameter FrPWakeupChannel

Channel used by the node to send a wakeup pattern.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FR_CHANNEL_A
literals	['FR_CHANNEL_A', 'FR_CHANNEL_B']

4.64 Parameter FrPWakeupPattern

Number of repetitions of the wakeup symbol that are combined to form a wakeup pattern when the node enters the POC:wakeup send state.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	63
max	63
min	0

4.65 Parameter FrPdAcceptedStartupRange

Expanded range of measured clock deviation allowed for startup frames during integration [Microticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	110
max	2743
min	0

4.66 Parameter FrPdListenTimeout

Value for the startup listen timeout and wakeup listen timeout. Although this is a node local parameter, the real time equivalent of this value should be the same for all nodes in the cluster [Microticks].

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	401202
max	2567692
min	1284

4.67 Parameter FrPdMicrotick

Duration of a microtick.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	T25NS
literals	['T100NS', 'T12_5NS', 'T200NS', 'T25NS', 'T50NS']

4.68 Reference FrCtrlEcucPartitionRef

Maps one single Flexray controller to zero or one ECUC partitions.

The ECUC partition referenced is a subset of the ECUC partitions where the Flexray driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.69 Container VendorSpecific

Freescale FlexRay module specific information.

Included subcontainers:

- [FrTestModeEnable](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.70 Parameter CCBaseAddress

Vendor specific:

Base address of the FlexRay module. It can be in 32-bit hexadecimal format.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	4293197824
max	4294967295
min	0

4.71 Parameter SystemFrequency

Vendor specific:

Set system bus clock, which is used as the clock of the CHI module (fCHI) in the FlexRay Controller.

This value is required for the TIMEOUT value in the SYMATOR register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	180
max	200
min	32

4.72 Parameter SystemMemoryAccessTimeOut

Vendor specific:

Configure TIMEOUT value in the System Memory Access Time-Out (FR_SYMATOR) register.

The TIMEOUT value in the FR_SYMATOR register and the CHI clock frequency fCHI in [MHz] must fulfill equation:

$$0 \leq \text{SYMATOR}[\text{TIMEOUT}] \leq [0.45 * \text{fCHI} - 8]$$

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	10
max	82
min	0

4.73 Parameter ChannelBitrate

Vendor specific:

FlexRay Bus Bit Rate.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	br_10Mbps
literals	['br_10Mbps', 'br_5Mbps', 'br_2500kbps']

4.74 Parameter SingleChannelModeEnabled

Vendor specific:

Enabling of single channel mode.

"false" - dual channel mode.

"true" - single channel mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.75 Parameter ForceReconfiguration

Vendor specific:

Tresos Configuration Plug-in

Force reconfiguration for this multiple configuration. To force reconfiguration "Buffer Reconfiguration" must be enabled by node FrPrepareLPduSupport

"true" - reconfiguration will be performed no matter if it is necessary or not.

"false" - reconfiguration will be not performed if it is not necessary.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.76 Container FrTestModeEnable

Vendor specific:

This container enables the generator test mode.

In this mode, generator templates calculates only with defined number of available physical resources.

It enables to test the buffers reconfiguration functionality with limited number of virtual buffer configurations.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.77 Parameter FrTestModeNumberOfResources

This field defines the number of available physical transmit and receive resources in the generator test mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	10
max	132
min	2

4.78 Container FrAbsoluteTimer

Specifies the absolute timer configuration parameters of the Fr.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.79 Parameter FrAbsTimerIdx

Contains the index of an absolute timer contained in Fr on a certain FlexRay CC.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	254
min	0

4.80 Container FrControllerDemEventParameterRefs

Container for the references to DemEventParameter elements which shall be invoked using the API Dem_ReportErrorStatus API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.81 Reference FR_E_CTRL_TESTRESULT

Reference to DEM event Id that is reported for FlexRay

Property	Value
type	ECUC-REFERENCE-DEF

Property	Value
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcucDefs/Dem/DemConfigSet/DemEventParameter

4.82 Container FrFifo

One First In First Out (FIFO) queued receive structure, defining the admittance criteria to the FIFO, and mandating the ability to admit messages into the FIFO based on Message Id filtering criteria.

Included subcontainers:

- [FrRange](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.83 Parameter FrAdmitWithoutMessageId

Determines whether or not frames received in the dynamic segment that do not contain a message ID will be admitted into the FIFO.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.84 Parameter FrBaseCycle

FIFO cycle counter acceptance criteria.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	63
min	0

4.85 Parameter FrChannels

FIFO channel admittance criteria.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FR_CHANNEL_A
literals	['FR_CHANNEL_A', 'FR_CHANNEL_B']

4.86 Parameter FrCycleRepetition

FIFO cycle counter acceptance criteria. Valid values are 1,2,4,5,8,10,16,20,32,40,50,64.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	64
min	1

4.87 Parameter FrFifoDepth

Fifo Depth.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	32
max	255
min	1

4.88 Parameter FrMsgIdMask

FIFO message identifier acceptance criteria (Mask filter). Value 0 disables the message ID filtering.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	65535
min	0

4.89 Parameter FrMsgIdMatch

FIFO message identifier acceptance criteria (Match filter).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0

Property	Value
max	65535
min	0

4.90 Container FrRange

FIFO Frame Id range acceptance criteria.

Included subcontainers:

- [FrVendorSpecific](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	4
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.91 Parameter FrRangeMin

First FrameId of this range that will be accepted/rejected by the FIFO.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	2047
min	0

4.92 Parameter FrRangeMax

Last FrameId of this range that will be accepted/rejected by the FIFO.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	2047
min	0

4.93 Container FrVendorSpecific

Vendor specific:

Freescale FlexRay module FIFO specific information.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.94 Parameter FrRangeMode

Vendor specific:

Selects whether the frames in the range are accepted to or rejected from the FIFO.

The rejection filters are applied at first and all frames within their ranges are not received by the FIFO.

Then the acceptance filters are applied and only frames within their ranges are received by the FIFO.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FR_ACCEPTANCE
literals	['FR_ACCEPTANCE', 'FR_REJECTION']

This chapter describes the Tresos configuration plug-in for the Flexray Driver. The most of the parameters are described below.



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Flexray	61
FR_DRIVER	112

Chapter 6

Module Documentation

6.1 Flexray

6.1.1 Detailed Description

Data Structures

- struct [Flexray_Ip_UserCallbackType](#)
User callback function. [More...](#)
- struct [Flexray_Ip_IndMemConfigType](#)
Structure to configure Segment 1, 2 size. [More...](#)
- struct [Flexray_Ip_FrameHeaderType](#)
Structure for all field of header. [More...](#)
- struct [Flexray_Ip_IndConfigType](#)
Structure to configure header frame, function and position of message buffer,. [More...](#)
- struct [Flexray_Ip_ShadowConfigType](#)
Structure to configure shadow buffer (physical message buffer index and position) [More...](#)
- struct [Flexray_Ip_RangeFilterType](#)
Structure for configure range filter. [More...](#)
- struct [Flexray_Ip_FifoConfigType](#)
Structure for configure rx FIFO (size, filter and watermark) [More...](#)
- struct [Flexray_Ip_CounterConfigType](#)
Structure for configure counter. [More...](#)
- struct [Flexray_Ip_CcConfigType](#)
Structure for configure communication controller. [More...](#)
- struct [Flexray_Ip_PeConfigType](#)
Flexray protocol engine configuration The structure is used as input parameters to configure all information of protocol engine. Parameters with prefix "g" must have same value in all nodes of a cluster. Parameters with prefix "p" may have different values in different nodes of a cluster. [More...](#)
- struct [Flexray_Ip_BufferStateType](#)
Structure for internal use. This structure is used by the driver for its internal logic. It must be provided by the application through the initialize functions, then it cannot be freed until the driver is de-initialized using Deinit functions. The application should make no assumptions about the content of this structure. [More...](#)

- struct [Flexray_Ip_StateType](#)
Structure for state of each instance. [More...](#)
- struct [Flexray_Ip_MtsFilterType](#)
Structure for filter cycle counter on Media access test symbol. [More...](#)
- struct [Flexray_Ip_ConfigType](#)
Structure for flexray user configuration. [More...](#)
- struct [Flexray_Ip_StateAllocateType](#)
Structure for allocate state User needs to declare a variable with [Flexray_Ip_StateAllocateType](#) type for initialize and assigned state. [More...](#)
- struct [Flexray_Ip_TimerConfigType](#)
Structure for configure timer. [More...](#)

Types Reference

- typedef void(* [Flexray_Ip_CallbackType](#)) (void *callbackParam)
User irq.
- typedef void(* [Flexray_Ip_PrCallbackType](#)) (uint32 prIntFlagValue, void *callbackParam)
Callback for protocol interrupt.
- typedef void(* [Flexray_Ip_ChiCallbackType](#)) (uint16 chiErrorValue, void *callbackParam)
Callback for CHI error interrupt.
- typedef void(* [Flexray_Ip_IndCallbackType](#)) (uint8 bufferId, void *callbackParam)
Callback for rx/tx message buffer interrupt.

Enum Reference

- enum [Flexray_Ip_StatusType](#)
- enum [Flexray_Ip_ChiCmdType](#)
Protocol Control Command.
- enum [Flexray_Ip_PocStateType](#)
Flexray Protocol State.
- enum [Flexray_Ip_ExtClkCorrectType](#)
External Offset Correction Application Using to trigger the application of the external offset correction value defined in the Protocol Configuration.
- enum [Flexray_Ip_GlobalIntType](#)
Global interrupt flags.
- enum [Flexray_Ip_WakeupStatusType](#)
Flexray Wakeup Status.
- enum [Flexray_Ip_StartupStatusType](#)
Flexray Startup Status, pegged to the AutoSAR enum, but uses SDK element names.
- enum [Flexray_Ip_ProtocolIntType](#)
Protocol Interrupt Flags.
- enum [Flexray_Ip_ChiIntType](#)
CHI Error Flags.
- enum [Flexray_Ip_WakeUpChannelType](#)
Choose wakeup for channel.

- enum [Flexray_Ip_MemLayoutType](#)
Layout for memory message buffer.
- enum [Flexray_Ip_RangeFilterModeType](#)
Enum for range filter type.
- enum [Flexray_Ip_CounterIncType](#)
Enum for counter increment type.
- enum [Flexray_Ip_TimerType](#)
Enum for choose timer.
- enum [Flexray_Ip_BitRateType](#)
FlexRay Bus Bit Rate.
- enum [Flexray_Ip_SoftwareStatesType](#)
FlexRay software states.

Function Reference

- [Flexray_Ip_PocStateType Flexray_Ip_GetCurrentState](#) (uint8 instNum)
Return current protocol state.
- [Flexray_Ip_StatusType Flexray_Ip_WaitPOCState](#) (uint8 instNum, [Flexray_Ip_PocStateType](#) pocState)
Waits for the POC to change its state.
- void [Flexray_Ip_ConfigTimer](#) (uint8 instNum, const [Flexray_Ip_TimerConfigType](#) *timerCfg)
Config timer operation.
- void [Flexray_Ip_StartTimer](#) (uint8 instNum, [Flexray_Ip_TimerType](#) timer)
Start timer.
- void [Flexray_Ip_StopTimer](#) (uint8 instNum, [Flexray_Ip_TimerType](#) timer)
Stop timer.
- void [Flexray_Ip_SetStatusCounter](#) (uint8 instNum, uint8 counterIndex, const [Flexray_Ip_CounterConfigType](#) *counterConfig)
Setup status counter.
- uint16 [Flexray_Ip_GetStatusCounter](#) (uint8 instNum, uint8 counterIndex)
Get status counter.
- void [Flexray_Ip_SetProtocolInterrupt](#) (uint8 instNum, [Flexray_Ip_ProtocolIntType](#) pInt, boolean enable)
Sets the chosen protocol interrupt.
- boolean [Flexray_Ip_GetProtocolInterruptFlag](#) (uint8 instNum, [Flexray_Ip_ProtocolIntType](#) pInt)
Gets the chosen protocol interrupt flag value.
- void [Flexray_Ip_ClearProtocolInterruptFlag](#) (uint8 instNum, [Flexray_Ip_ProtocolIntType](#) pInt)
Clears the chosen protocol interrupt flag.
- boolean [Flexray_Ip_GetGlobalInterruptFlag](#) (uint8 instNum, [Flexray_Ip_GlobalIntType](#) gInt)
Gets the chosen global interrupt flag value.
- void [Flexray_Ip_SetGlobalInterrupt](#) (uint8 instNum, [Flexray_Ip_GlobalIntType](#) gInt, boolean enable)
Sets the chosen global interrupt.
- void [Flexray_Ip_ClearGlobalInterruptFlag](#) (uint8 instNum, [Flexray_Ip_GlobalIntType](#) gInt)
Clears the chosen global interrupt flag.
- boolean [Flexray_Ip_GetCHIInterruptFlag](#) (uint8 instNum, [Flexray_Ip_ChiIntType](#) cInt)
Gets the chosen controller host interface interrupt flag value.
- void [Flexray_Ip_ClearCHIInterruptFlag](#) (uint8 instNum, [Flexray_Ip_ChiIntType](#) cInt)
Clears the chosen controller host interface interrupt flag.

- void [Flexray_Ip_SetStatusMonitorSlot](#) (uint8 instNum, uint8 monitorIndex, uint16 slotNumber)
Setup the slot status monitor.
- uint8 [Flexray_Ip_GetStatusMonitor](#) (uint8 instNum, uint8 monitorIndex, boolean channelA, boolean even←Cycle)
Get slot status from slot status monitor.
- void [Flexray_Ip_SetExternalCorrection](#) (uint8 instNum, boolean rateCorrect, [Flexray_Ip_ExtClkCorrectType](#) correctType)
Apply external rate or offset correction value.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_SetMTS](#) (uint8 instNum, boolean channelA, boolean enable)
Enables or disables the generation of the MTS on either channel.
- uint16 [Flexray_Ip_GetCurrentSlotCounter](#) (uint8 instNum, boolean channelA)
Get current slot counter on channel.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_Init](#) (uint8 instNum, const [Flexray_Ip_ConfigType](#) *config, const [Flexray_Ip_StateAllocateType](#) *stateAlloc)
Initialize CC, PE, message buffers and assign user callback.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_Deinit](#) (uint8 instNum)
Execute shutdown sequence.
- void [Flexray_Ip_ReceiveInd](#) (uint8 instNum, uint8 bufferId, [Flexray_Ip_FrameHeaderType](#) *frameHdr, uint8 *pData, uint8 dataLength)
Receive an individual buffer. Return immediately.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_Send](#) (uint8 instNum, uint8 bufferId, const uint8 *pData, uint8 data←Length)
Send an individual buffer. Return immediately.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_SendBlocking](#) (uint8 instNum, uint8 bufferId, const uint8 *pData, uint8 dataLength, uint32 timeout_ms)
Send an individual buffer, wait until the message is sent or time out.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_ReceiveIndBlocking](#) (uint8 instNum, uint8 bufferId, [Flexray_Ip_FrameHeaderType](#) *frameHdr, uint8 *pData, uint8 dataLength, uint32 timeout_ms)
Receive an individual buffer. Wait until message is received or time out.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_ReceiveFIFO](#) (uint8 instNum, boolean channel, [Flexray_Ip_FrameHeaderType](#) *frameHdr, uint8 *pData, uint8 dataLength)
Receive a buffer from fifo. Return immediately.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_GetTransferStatus](#) (uint8 instNum, uint8 bufferId)
Get status of an individual buffer transfer.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_Abort](#) (uint8 instNum, uint8 bufferId)
Abort transfer of an individual buffer. For Rx, If abort function is successful, memory of this buffer will be not updated about frame header and payload. For Tx, If abort function is successful, tx message buffer will be not sent on bus. If the application want to send/receive message buffer, please call again receive/send function.
- [Flexray_Ip_StartupStatusType](#) [Flexray_Ip_GetStartupStatus](#) (uint8 instNum)
Return startup status.
- uint8 [Flexray_Ip_GetCurrentCycleCount](#) (uint8 instNum)
Return current cycle count.
- [Flexray_Ip_WakeupChannelType](#) [Flexray_Ip_GetWakeupChannel](#) (uint8 instNum)
Return wakeup channel.
- [Flexray_Ip_WakeupStatusType](#) [Flexray_Ip_GetWakeupStatus](#) (uint8 instNum)
Return wakeup status.
- void [Flexray_Ip_GetDefaultConfig](#) (const [Flexray_Ip_ConfigType](#) *config)
Get PE and CC default configuration.

- void [Flexray_Ip_SetBasicDynamicTxFrame](#) (uint8 instNum, uint8 bufferId, boolean PPI, uint8 payloadLength)

Set PPI or payloadLength for Dynamic frame in individual buffer failed.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_ReConfigIndividual](#) (uint8 instNum, [Flexray_Ip_IndConfigType](#) *pIndConfig)

Reconfig an individual buffer. Return immediately. The field phyMBIndex in struct [Flexray_Ip_IndConfigType](#) cannot be reconfigured. The field MBIndex in struct [Flexray_Ip_IndConfigType](#) cannot be reconfigured.
- [Flexray_Ip_StatusType](#) [Flexray_Ip_GetNMVector](#) (uint8 instNum, uint8 *data, uint8 *configLen)

Return network management vector of last cycle.
- uint8 [Flexray_Ip_GetFifoLevel](#) (uint8 instNum, boolean isChnB)

Return number of entries in fifo.
- void [FLEXRAY_SetFiFoRangeFilter](#) (FLEXRAY_Type *base, uint8 filterNum, const [Flexray_Ip_RangeFilterType](#) *filterCfg)

Set FiFo Range Filter.
- void [FLEXRAY_InitPE](#) (uint8 instNum, const [Flexray_Ip_PeConfigType](#) *peConfig)

Set the protocol engine configuration.
- void [FLEXRAY_ConfigureTimer1](#) (FLEXRAY_Type *base, const [Flexray_Ip_TimerConfigType](#) *timerCfg)

Configure timer 1.
- void [FLEXRAY_ConfigureTimer2](#) (FLEXRAY_Type *base, const [Flexray_Ip_TimerConfigType](#) *timerCfg)

Configure timer 2.

6.1.2 Data Structure Documentation

6.1.2.1 struct Flexray_Ip_UserCallbackType

User callback function.

Definition at line 545 of file Flexray_Ip_Types.h.

Data Fields

- [Flexray_Ip_PrCallbackType](#) protocolCallback
- void * [prCallbackParam](#)
- [Flexray_Ip_ChiCallbackType](#) chiCallback
- void * [chiCallbackParam](#)
- [Flexray_Ip_CallbackType](#) wakeupCallback
- void * [wakeupCallbackParam](#)
- [Flexray_Ip_CallbackType](#) fifoACallback
- void * [fifoACallbackParam](#)
- [Flexray_Ip_CallbackType](#) fifoBCallback
- void * [fifoBCallbackParam](#)
- [Flexray_Ip_IndCallbackType](#) rxMBCallback
- void * [rxCallbackParam](#)
- [Flexray_Ip_IndCallbackType](#) txMBCallback
- void * [txCallbackParam](#)

6.1.2.1.1 Field Documentation

6.1.2.1.1.1 **protocolCallback** `Flexray_Ip_PrCallbackType` protocolCallback

Callback when protocol interrupt occurs

Definition at line 547 of file Flexray_Ip_Types.h.

6.1.2.1.1.2 **prCallbackParam** `void*` prCallbackParam

Parameter passed to the protocol callback

Definition at line 548 of file Flexray_Ip_Types.h.

6.1.2.1.1.3 **chiCallback** `Flexray_Ip_ChiCallbackType` chiCallback

Callback when chi error interrupt occurs

Definition at line 549 of file Flexray_Ip_Types.h.

6.1.2.1.1.4 **chiCallbackParam** `void*` chiCallbackParam

Parameter passed to the chi callback

Definition at line 550 of file Flexray_Ip_Types.h.

6.1.2.1.1.5 **wakeupCallback** `Flexray_Ip_CallbackType` wakeupCallback

Callback when wakeup interrupt occurs

Definition at line 551 of file Flexray_Ip_Types.h.

6.1.2.1.1.6 **wakeupCallbackParam** `void*` wakeupCallbackParam

Parameter passed to the chi callback

Definition at line 552 of file Flexray_Ip_Types.h.

6.1.2.1.1.7 `fifoACallback` `Flexray_Ip_CallbackType` `fifoACallback`

Callback when receive FIFO Channel A Almost Full interrupt occurs

Definition at line 553 of file `Flexray_Ip_Types.h`.

6.1.2.1.1.8 `fifoACallbackParam` `void*` `fifoACallbackParam`

Parameter passed to the FIFO A callback

Definition at line 554 of file `Flexray_Ip_Types.h`.

6.1.2.1.1.9 `fifoBCallback` `Flexray_Ip_CallbackType` `fifoBCallback`

Callback when receive FIFO Channel B Almost Full interrupt occurs

Definition at line 555 of file `Flexray_Ip_Types.h`.

6.1.2.1.1.10 `fifoBCallbackParam` `void*` `fifoBCallbackParam`

Parameter passed to the FIFO B callback

Definition at line 556 of file `Flexray_Ip_Types.h`.

6.1.2.1.1.11 `rxMBCallback` `Flexray_Ip_IndCallbackType` `rxMBCallback`

Callback when receive message buffer interrupt occurs

Definition at line 557 of file `Flexray_Ip_Types.h`.

6.1.2.1.1.12 `rxCallbackParam` `void*` `rxCallbackParam`

Parameter passed to the RX callback

Definition at line 558 of file `Flexray_Ip_Types.h`.

6.1.2.1.1.13 **txMBCallback** `Flexray_Ip_IndCallbackType txMBCallback`

Callback when transmit message buffer interrupt occurs

Definition at line 559 of file Flexray_Ip_Types.h.

6.1.2.1.1.14 **txCallbackParam** `void* txCallbackParam`

Parameter passed to the TX callback

Definition at line 560 of file Flexray_Ip_Types.h.

6.1.2.2 **struct Flexray_Ip_IndMemConfigType**

Structure to configure Segment 1, 2 size.

Definition at line 568 of file Flexray_Ip_Types.h.

6.1.2.3 **struct Flexray_Ip_FrameHeaderType**

Structure for all field of header.

Definition at line 582 of file Flexray_Ip_Types.h.

6.1.2.4 **struct Flexray_Ip_IndConfigType**

Structure to configure header frame, function and position of message buffer,.

Definition at line 601 of file Flexray_Ip_Types.h.

Data Fields

- uint8 `MBIndex`
- uint16 `phyMBIndex`
- boolean `firstInit`
- uint16 `frameID`
- uint8 `payloadLength`
- boolean `isTx`
- boolean `channelA`
- boolean `channelB`
- boolean `ccFilter`
- uint8 `ccFilterVal`
- uint8 `ccFilterMask`
- boolean `repeatTx`
- boolean `enable`

6.1.2.4.1 Field Documentation

6.1.2.4.1.1 MBIndex `uint8 MBIndex`

Message buffer index (index of registers such as FR_MBCCSRn, FR_MBCCFRn ...)

Definition at line 603 of file Flexray_Ip_Types.h.

6.1.2.4.1.2 phyMBIndex `uint16 phyMBIndex`

Physical message buffer index

Definition at line 604 of file Flexray_Ip_Types.h.

6.1.2.4.1.3 firstInit `boolean firstInit`

Buffer should be used for the initialization

Definition at line 605 of file Flexray_Ip_Types.h.

6.1.2.4.1.4 frameID `uint16 frameID`

Slot ID in which frame is transmitted/received

Definition at line 606 of file Flexray_Ip_Types.h.

6.1.2.4.1.5 payloadLength `uint8 payloadLength`

Length in 2-byte unit, only for dynamic slot transmission

Definition at line 609 of file Flexray_Ip_Types.h.

6.1.2.4.1.6 isTx `boolean isTx`

True: Tx buffer, false: Rx buffer

Definition at line 610 of file Flexray_Ip_Types.h.

6.1.2.4.1.7 **channelA** `boolean channelA`

True: enable transfer on channel A, false otherwise

Definition at line 611 of file Flexray_Ip_Types.h.

6.1.2.4.1.8 **channelB** `boolean channelB`

True: enable transfer on channel B, false otherwise. Note that different combinations of channelA and channelB fields shall have different effects. Please see Channel Assignment Description table in RM for details

Definition at line 612 of file Flexray_Ip_Types.h.

6.1.2.4.1.9 **ccFilter** `boolean ccFilter`

True: enable cycle count filter, false otherwise

Definition at line 615 of file Flexray_Ip_Types.h.

6.1.2.4.1.10 **ccFilterVal** `uint8 ccFilterVal`

Cycle count filter value

Definition at line 616 of file Flexray_Ip_Types.h.

6.1.2.4.1.11 **ccFilterMask** `uint8 ccFilterMask`

Cycle count mask. This value is ANDed with current cycle count and compare with ccFilterVal

Definition at line 617 of file Flexray_Ip_Types.h.

6.1.2.4.1.12 **repeatTx** `boolean repeatTx`

Only used if this is a tx individual buffer. True: buffer is transmitted again automatically. False: buffer is transmitted only once.

Definition at line 618 of file Flexray_Ip_Types.h.

6.1.2.4.1.13 enable `boolean enable`

True: Enable this message buffer. False: Disable this message buffer

Definition at line 622 of file Flexray_Ip_Types.h.

6.1.2.5 struct Flexray_Ip_ShadowConfigType

Structure to configure shadow buffer (physical message buffer index and position)

Definition at line 633 of file Flexray_Ip_Types.h.

Data Fields

- uint8 [phyMBIndex](#)
- boolean [channelA](#)
- boolean [segment1](#)

6.1.2.5.1 Field Documentation**6.1.2.5.1.1 phyMBIndex** `uint8 phyMBIndex`

Physical message buffer index

Definition at line 635 of file Flexray_Ip_Types.h.

6.1.2.5.1.2 channelA `boolean channelA`

True: receive shadow buffer for channel A. False: receive shadow buffer for channel B

Definition at line 636 of file Flexray_Ip_Types.h.

6.1.2.5.1.3 segment1 `boolean segment1`

True: receive shadow buffer for segment 1. False: receive shadow buffer for segment 2. Note that each combination of channel and segment can be assigned only one shadow buffer

Definition at line 637 of file Flexray_Ip_Types.h.

6.1.2.6 struct Flexray_Ip_RangeFilterType

Structure for configure range filter.

Definition at line 646 of file Flexray_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	rangeFilterEnable	Range filter enable. True: Enable. False: Disable
Flexray_Ip_RangeFilterModeType	rangeFilterMode	Range filter mode
uint16	rangeFilterLower	Frame ID lower value for range filter
uint16	rangeFilterUpper	Frame ID upper value for range filter

6.1.2.7 struct Flexray_Ip_FifoConfigType

Structure for configure rx FIFO (size, filter and watermark)

Definition at line 660 of file Flexray_Ip_Types.h.

Data Fields

Type	Name	Description
Flexray_Ip_FifoChannelType	FifoChannel	
uint8	startIndex	Physical message buffer index of the first message buffer in fifo
uint8	fifoDepth	Number of entries in fifo. Zero value shall disable fifo
uint8	entrySize	Size of frame data section. Must be an even number
uint16	fidRejMask	FRAME ID rejection Filter Mask. This is ANDed with FRAME ID and compare with fidRejVal
uint16	fidRejVal	FRAME ID rejection Filter Mask
uint16	midAccMask	MESSAGE ID Acceptance Filter Mask. This is ANDed with MESSAGE ID and compare with fidAccVal
uint16	midAccVal	MESSAGE ID Acceptance Filter Value
Flexray_Ip_RangeFilterType	rangeFilter[4]	Frame ID range Filter. Maximum number of range filter is 4
uint8	watermark	Watermark value to used with channel A Almost Full Interrupt

6.1.2.8 struct Flexray_Ip_CounterConfigType

Structure for configure counter.

Definition at line 679 of file Flexray_Ip_Types.h.

Data Fields

- [Flexray_Ip_CounterIncType](#) counterInc
- boolean multiCycle
- boolean validFrameOnly
- boolean syncFrameOnly

- boolean [nullFrameOnly](#)
- boolean [startupFrameOnly](#)
- boolean [syntaxErr](#)
- boolean [contentErr](#)
- boolean [boundaryViolate](#)
- boolean [transmitConflict](#)

6.1.2.8.1 Field Documentation

6.1.2.8.1.1 **counterInc** [Flexray_Ip_CounterIncType](#) counterInc

Counter increment type

Definition at line 681 of file Flexray_Ip_Types.h.

6.1.2.8.1.2 **multiCycle** [boolean](#) multiCycle

True: count on all cycle. False: count on previous cycle only

Definition at line 682 of file Flexray_Ip_Types.h.

6.1.2.8.1.3 **validFrameOnly** [boolean](#) validFrameOnly

True: count valid frame only. False otherwise

Definition at line 683 of file Flexray_Ip_Types.h.

6.1.2.8.1.4 **syncFrameOnly** [boolean](#) syncFrameOnly

True: count sync frame only. False otherwise

Definition at line 684 of file Flexray_Ip_Types.h.

6.1.2.8.1.5 **nullFrameOnly** [boolean](#) nullFrameOnly

True: count valid frame only. False otherwise

Definition at line 685 of file Flexray_Ip_Types.h.

6.1.2.8.1.6 **startupFrameOnly** `boolean startupFrameOnly`

True: count startup frame only. False otherwise

Definition at line 686 of file Flexray_Ip_Types.h.

6.1.2.8.1.7 **syntaxErr** `boolean syntaxErr`

True: count slots with the syntax error indicator bit set to 1. False otherwise

Definition at line 687 of file Flexray_Ip_Types.h.

6.1.2.8.1.8 **contentErr** `boolean contentErr`

True: count slots with the content error indicator bit set to 1. False otherwise

Definition at line 688 of file Flexray_Ip_Types.h.

6.1.2.8.1.9 **boundaryViolate** `boolean boundaryViolate`

True: count slots with the boundary violation indicator bit set to 1. False otherwise

Definition at line 689 of file Flexray_Ip_Types.h.

6.1.2.8.1.10 **transmitConflict** `boolean transmitConflict`

True: count slots with the transmission conflict indicator bit set to 1. False otherwise

Definition at line 690 of file Flexray_Ip_Types.h.

6.1.2.9 **struct Flexray_Ip_CcConfigType**

Structure for configure communication controller.

Definition at line 698 of file Flexray_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	freezeMode	Transition to freeze mode or not(1/0) when system bus fail happen
boolean	singleChnlMode	Single Channel Device mode or not(true/false) True: single channel device mode False: dual channel device mode
boolean	channelA	True: use channel A/False: not use channel A
boolean	channelB	True: use channel B/False: not use channel A
boolean	syncFrameFilter	Synchronization frame filtering(true/false) True: Synchronization frame filtering enabled False: Synchronization frame filtering disabled
Flexray_Ip_BitRateType	bitRate	This field defines the FlexRay Bus Bit Rate
Flexray_Ip_MemLayoutType	memLayout	Controls the location of the system memory base address for the FIFOs
uint8	timeout	System memory access timeout
uint8 *	pSysMem	Pointer to system memory area User located memory for flexray module to use. The size of this memory block is calculated by: (number of S1 buffers + number of S2 buffers + number of shadow buffer + fifo depth) * 8 + (number of S1 buffers + number of S1 shadow buffer) * S1 buffer size + (number of S2 buffers + number of S2 shadow buffer) * S2 buffer size)
uint8 *	pFiFoSysMem	Only used if memLayout is FLEXRAY_MEM_LAYOUT_DEDICATED and pointer to fifo system memory area

6.1.2.10 struct Flexray_Ip_PeConfigType

Flexray protocol engine configuration The structure is used as input parameters to configure all information of protocol engine. Parameters with prefix "g" must have same value in all nodes of a cluster. Parameters with prefix "p" may have different values in different nodes of a cluster.

Definition at line 728 of file Flexray_Ip_Types.h.

Data Fields

- uint8 [gColdstartAttempts](#)
- uint8 [gdActionPointOffset](#)
- uint8 [gdCASRxLowMax](#)
- uint8 [gdDynamicSlotIdlePhase](#)
- uint8 [gdMinislotActionPointOffset](#)
- uint16 [gdStaticSlot](#)
- boolean [gSymbolWindowExists](#)
- uint8 [gdTSSTransmitter](#)
- uint8 [gdWakeupSymbolRxIdle](#)
- uint8 [gdWakeupSymbolRxLow](#)

- uint16 gdWakeupSymbolRxWindow
- uint8 gdWakeupSymbolTxIdle
- uint8 gdWakeupSymbolTxLow
- uint16 gMacroPerCycle
- boolean gMiniSlotExists
- uint16 gOffsetCorrectionStart
- uint8 gPayloadLengthStatic
- uint8 gSyncNodeMax
- uint32 pdListenTimeout
- uint8 pMacroInitialOffsetA
- uint8 pMacroInitialOffsetB
- uint8 pPayloadLengthDynMax
- uint8 pAllowPassiveToActive
- uint8 pClusterDriftDamping
- uint16 pKeySlotId
- boolean pKeySlotUsedForStartup
- boolean pKeySlotUsedForSync
- uint16 pLatestTx
- uint32 pMicroPerCycle
- uint16 pRateCorrectionOut
- uint16 pOffsetCorrectionOut
- boolean pSingleSlotEnabled
- boolean pWakeupChannel
- uint8 pWakeupPattern
- uint16 keySlotHeaderCrc
- uint8 pExternOffsetCorrection
- uint8 pExternRateCorrection
- uint8 gNetworkManagementVectorLength

6.1.2.10.1 Field Documentation

6.1.2.10.1.1 gColdstartAttempts uint8 gColdstartAttempts

Number of coldstart node in network (Range: 2->31)

Definition at line 730 of file Flexray_Ip_Types.h.

6.1.2.10.1.2 gdActionPointOffset uint8 gdActionPointOffset

Number of macroticks the action point is offset from the beginning of a static slot or symbol window; value into range: 1-63MT.

Definition at line 731 of file Flexray_Ip_Types.h.

6.1.2.10.1.3 gdCASRxLowMax uint8 gdCASRxLowMax

Upper limit of the CAS acceptance window; value into range: 67-99 gdBit

Definition at line 733 of file Flexray_Ip_Types.h.

6.1.2.10.1.4 gdDynamicSlotIdlePhase uint8 gdDynamicSlotIdlePhase

Duration of the idle phase within a dynamic slot. from 0 to 2

Definition at line 734 of file Flexray_Ip_Types.h.

6.1.2.10.1.5 gdMinislotActionPointOffset uint8 gdMinislotActionPointOffset

Number of macroticks the minislot action point is offset from the beginning of a minislot value into range: 1-31MT

Definition at line 736 of file Flexray_Ip_Types.h.

6.1.2.10.1.6 gdStaticSlot uint16 gdStaticSlot

Duration of a static slot (4->661 MT)

Definition at line 738 of file Flexray_Ip_Types.h.

6.1.2.10.1.7 gSymbolWindowExists boolean gSymbolWindowExists

Enable/Disable symbol window

Definition at line 739 of file Flexray_Ip_Types.h.

6.1.2.10.1.8 gdTSSTransmitter uint8 gdTSSTransmitter

number of bits in the Transmission Start Sequence(3-15gdBit)

Definition at line 741 of file Flexray_Ip_Types.h.

6.1.2.10.1.9 **gdWakeupSymbolRxIdle** `uint8 gdWakeupSymbolRxIdle`

Number of bits used by the node to test the duration of the 'idle' portion of a received wakeup symbol (14-59gdBit)

Definition at line 742 of file Flexray_Ip_Types.h.

6.1.2.10.1.10 **gdWakeupSymbolRxLow** `uint8 gdWakeupSymbolRxLow`

Number of bits used by the node to test the LOW portion of a received wakeup symbol (11-59gdBit)

Definition at line 744 of file Flexray_Ip_Types.h.

6.1.2.10.1.11 **gdWakeupSymbolRxWindow** `uint16 gdWakeupSymbolRxWindow`

The size of the window used to detect wakeup (76-301 gdBit)

Definition at line 746 of file Flexray_Ip_Types.h.

6.1.2.10.1.12 **gdWakeupSymbolTxIdle** `uint8 gdWakeupSymbolTxIdle`

Number of bits used by the node to transmit the 'idle' part of a wakeup symbol (45-180 gdBit)

Definition at line 747 of file Flexray_Ip_Types.h.

6.1.2.10.1.13 **gdWakeupSymbolTxLow** `uint8 gdWakeupSymbolTxLow`

Number of bits used by the node to transmit the LOW part of a wakeup symbol (15-60 gdBit)

Definition at line 748 of file Flexray_Ip_Types.h.

6.1.2.10.1.14 **gMacroPerCycle** `uint16 gMacroPerCycle`

Number of macroticks in a communication cycle (10 - 16000MT)

Definition at line 750 of file Flexray_Ip_Types.h.

6.1.2.10.1.15 gMiniSlotExists `boolean gMiniSlotExists`

disable/enable minislot

Definition at line 753 of file Flexray_Ip_Types.h.

6.1.2.10.1.16 gOffsetCorrectionStart `uint16 gOffsetCorrectionStart`

Start of the offset correction phase within the NIT, expressed as the number of macroticks from the start of cycle (9 - 15999 MT) Note that this value should be greater than $gdStaticSlot * gNumberOfStaticSlots + gdMinislot * gNumberOfMinislots + gdSymbolWindow$

Definition at line 756 of file Flexray_Ip_Types.h.

6.1.2.10.1.17 gPayloadLengthStatic `uint8 gPayloadLengthStatic`

Payload length of a static frame

Definition at line 760 of file Flexray_Ip_Types.h.

6.1.2.10.1.18 gSyncNodeMax `uint8 gSyncNodeMax`

Maximum number of nodes that may send frames with the sync frame indicator bit set to one.

Definition at line 761 of file Flexray_Ip_Types.h.

6.1.2.10.1.19 pdListenTimeout `uint32 pdListenTimeout`

Value for the startup listen timeout and wakeup listen timeout

Definition at line 762 of file Flexray_Ip_Types.h.

6.1.2.10.1.20 pMacroInitialOffsetA `uint8 pMacroInitialOffsetA`

2-68MT

Definition at line 763 of file Flexray_Ip_Types.h.

6.1.2.10.1.21 **pMacroInitialOffsetB** uint8 pMacroInitialOffsetB

2-68MT

Definition at line 764 of file Flexray_Ip_Types.h.

6.1.2.10.1.22 **pPayloadLengthDynMax** uint8 pPayloadLengthDynMax

Maximum payload length for dynamic frames

Definition at line 765 of file Flexray_Ip_Types.h.

6.1.2.10.1.23 **pAllowPassiveToActive** uint8 pAllowPassiveToActive

If set to zero, the CC is not allowed to transition from normal passive to normal active

Definition at line 767 of file Flexray_Ip_Types.h.

6.1.2.10.1.24 **pClusterDriftDamping** uint8 pClusterDriftDamping

Local cluster drift damping factor used for rate correction (0-20uT)

Definition at line 768 of file Flexray_Ip_Types.h.

6.1.2.10.1.25 **pKeySlotId** uint16 pKeySlotId

ID of the slot used to transmit the startup frame, sync frame, or designated single slot frame

Definition at line 772 of file Flexray_Ip_Types.h.

6.1.2.10.1.26 **pKeySlotUsedForStartup** boolean pKeySlotUsedForStartup

key slot is startup frame or not ?

Definition at line 773 of file Flexray_Ip_Types.h.

6.1.2.10.1.27 pKeySlotUsedForSync `boolean pKeySlotUsedForSync`

key slot is sync frame or not ?

Definition at line 774 of file Flexray_Ip_Types.h.

6.1.2.10.1.28 pLatestTx `uint16 pLatestTx`

Number of the last minislot in which a frame transmission can start in the dynamic segment

Definition at line 775 of file Flexray_Ip_Types.h.

6.1.2.10.1.29 pMicroPerCycle `uint32 pMicroPerCycle`

Nominal number of microticks in the communication cycle of the local node (640-640000 uT)

Definition at line 778 of file Flexray_Ip_Types.h.

6.1.2.10.1.30 pRateCorrectionOut `uint16 pRateCorrectionOut`

Magnitude of the maximum permissible rate correction value (2-1923 uT)

Definition at line 781 of file Flexray_Ip_Types.h.

6.1.2.10.1.31 pOffsetCorrectionOut `uint16 pOffsetCorrectionOut`

Magnitude of the maximum permissible offset correction value (13-15567 uT)

Definition at line 782 of file Flexray_Ip_Types.h.

6.1.2.10.1.32 pSingleSlotEnabled `boolean pSingleSlotEnabled`

Flag indicating whether or not the node shall enter single slot mode following startup

Definition at line 783 of file Flexray_Ip_Types.h.

6.1.2.10.1.33 **pWakeupChannel** `boolean pWakeupChannel`

Channel used by the node to send a wakeup pattern: false -> A, true -> B

Definition at line 784 of file Flexray_Ip_Types.h.

6.1.2.10.1.34 **pWakeupPattern** `uint8 pWakeupPattern`

Number of repetitions of the wakeup symbol that are combined to form a wakeup pattern when the node enters the POC:wakeup send state

Definition at line 785 of file Flexray_Ip_Types.h.

6.1.2.10.1.35 **keySlotHeaderCrc** `uint16 keySlotHeaderCrc`

header CRC for key slot (0x00-0x7FF)

Definition at line 788 of file Flexray_Ip_Types.h.

6.1.2.10.1.36 **pExternOffsetCorrection** `uint8 pExternOffsetCorrection`

Number of microticks added or subtracted to the NIT to carry out a host-requested external offset correction (0-7uT)

Definition at line 789 of file Flexray_Ip_Types.h.

6.1.2.10.1.37 **pExternRateCorrection** `uint8 pExternRateCorrection`

Number of microticks added or subtracted to the cycle to carry out a host-requested external rate correction (0-7uT)

Definition at line 790 of file Flexray_Ip_Types.h.

6.1.2.10.1.38 **gNetworkManagementVectorLength** `uint8 gNetworkManagementVectorLength`

The length of the NMVector in byte

Definition at line 791 of file Flexray_Ip_Types.h.

6.1.2.11 struct Flexray_Ip_BufferStateType

Structure for internal use. This structure is used by the driver for its internal logic. It must be provided by the application through the initialize functions, then it cannot be freed until the driver is de-initialized using Deinit functions. The application should make no assumptions about the content of this structure.

Definition at line 802 of file Flexray_Ip_Types.h.

6.1.2.12 struct Flexray_Ip_StateType

Structure for state of each instance.

Definition at line 821 of file Flexray_Ip_Types.h.

6.1.2.13 struct Flexray_Ip_MtsFilterType

Structure for filter cycle counter on Media access test symbol.

Definition at line 838 of file Flexray_Ip_Types.h.

6.1.2.14 struct Flexray_Ip_ConfigType

Structure for flexray user configuration.

Definition at line 849 of file Flexray_Ip_Types.h.

Data Fields

- uint16 [fifoTimerDuration](#)
- boolean [timer2IsRelative](#)
- [Flexray_Ip_MtsFilterType](#) * [mtsfilterChnA](#)
- [Flexray_Ip_MtsFilterType](#) * [mtsfilterChnB](#)

6.1.2.14.1 Field Documentation

6.1.2.14.1.1 fifoTimerDuration uint16 fifoTimerDuration

Timer duration in macroticks. See FIFO Periodic Timer section for details

Definition at line 860 of file Flexray_Ip_Types.h.

6.1.2.14.1.2 **timer2IsRelative** `boolean timer2IsRelative`

True: TIMER2 is relative. False: TIMER2 is absolute

Definition at line 861 of file Flexray_Ip_Types.h.

6.1.2.14.1.3 **mtsfilterChnA** `Flexray_Ip_MtsFilterType* mtsfilterChnA`

MTS cycle filter for Channel A

Definition at line 862 of file Flexray_Ip_Types.h.

6.1.2.14.1.4 **mtsfilterChnB** `Flexray_Ip_MtsFilterType* mtsfilterChnB`

MTS cycle filter for Channel B

Definition at line 863 of file Flexray_Ip_Types.h.

6.1.2.15 **struct Flexray_Ip_StateAllocateType**

Structure for allocate state User needs to declare a variable with `Flexray_Ip_StateAllocateType` type for initialize and assigned state.

Definition at line 873 of file Flexray_Ip_Types.h.

6.1.2.16 **struct Flexray_Ip_TimerConfigType**

Structure for configure timer.

Definition at line 885 of file Flexray_Ip_Types.h.

6.1.3 Types Reference

6.1.3.1 **Flexray_Ip_CallbackType**

```
typedef void(* Flexray_Ip_CallbackType) (void *callbackParam)
```

User irq.

The individual tx, rx interrupts will be managed by driver. The other interrupt will be managed by user, user will use provided set/get/clear function for these interrupt.

Callback for wakeup ,receive FIFO Channel A Almost Full receive FIFO Channel B Almost Full interrupt

param[in] callbackParam callback parameters for user

Definition at line 508 of file Flexray_Ip_Types.h.

6.1.3.2 Flexray_Ip_PrCallbackType

```
typedef void(* Flexray_Ip_PrCallbackType) (uint32 prIntFlagValue, void *callbackParam)
```

Callback for protocol interrupt.

param[in] prIntFlagValue Used for get all protocol interrupt flags. param[in] callbackParam callback parameters for user

Definition at line 518 of file Flexray_Ip_Types.h.

6.1.3.3 Flexray_Ip_ChiCallbackType

```
typedef void(* Flexray_Ip_ChiCallbackType) (uint16 chiErrorValue, void *callbackParam)
```

Callback for CHI error interrupt.

param[in] chiErrorValue Used for get all chi error flags. param[in] callbackParam callback parameters for user

Definition at line 528 of file Flexray_Ip_Types.h.

6.1.3.4 Flexray_Ip_IndCallbackType

```
typedef void(* Flexray_Ip_IndCallbackType) (uint8 bufferId, void *callbackParam)
```

Callback for rx/tx message buffer interrupt.

param[in] bufferid Used for process each message buffer. param[in] callbackParam callback parameters for user

Definition at line 538 of file Flexray_Ip_Types.h.

6.1.4 Enum Reference

Enumerator

6.1.4.1 Flexray_Ip_StatusType

```
enum Flexray_Ip_StatusType
```

Enumerator

FLEXRAY_STATUS_SUCCESS	Generic operation success status
FLEXRAY_STATUS_ERROR	Generic operation failure status
FLEXRAY_STATUS_BUSY	Generic operation busy status
FLEXRAY_STATUS_TIMEOUT	Generic operation timeout status
FLEXRAY_STATUS_UNSUPPORTED	Generic operation unsupported status
FLEXRAY_STATUS_ABORTED	FLEXRAY aborted status
FLEXRAY_STATUS_TX_CONFLICT_A	FLEXRAY TX conflict reported by hardware on channel A
FLEXRAY_STATUS_TX_CONFLICT_B	FLEXRAY TX conflict reported by hardware on channel B
FLEXRAY_STATUS_TX_CONFLICT_AB	FLEXRAY TX conflict reported by hardware on channel A & B
FLEXRAY_STATUS_FIFO_A_ERROR_CONFIG	Start index is not equal than read index into FIFO A when initialize again
FLEXRAY_STATUS_FIFO_B_ERROR_CONFIG	Start index is not equal than read index into FIFO B when initialize again
FLEXRAY_STATUS_FIFO_AB_ERROR_CONFIG	Start index is not equal than read index into FIFO A & B when initialize again

Definition at line 179 of file Flexray_Ip_Types.h.

6.1.4.2 Flexray_Ip_ChiCmdType

```
enum Flexray_Ip_ChiCmdType
```

Protocol Control Command.

Enumerator

FLEXRAY_CHI_CMD_ALLOW_COLDSTART	Active cold start cluster
FLEXRAY_CHI_CMD_ALL_SLOTS	Delayed transition to the all slots transmission mode
FLEXRAY_CHI_CMD_CONFIG	Transition to the POC:config state
FLEXRAY_CHI_CMD_FREEZE	Transition to POC:halt state
FLEXRAY_CHI_CMD_READY	Transition to POC:ready state
FLEXRAY_CHI_CMD_CONFIG_COMPLETE	Transition to POC:ready state

Enumerator

FLEXRAY_CHI_CMD_RUN	Transition to POC:startup start state
FLEXRAY_CHI_CMD_DEFAULT_CONFIG	Transition to POC:default config state
FLEXRAY_CHI_CMD_HALT	Delayed transition to the POC:halt state
FLEXRAY_CHI_CMD_WAKEUP	Initiate the wakeup procedure

Definition at line 217 of file Flexray_Ip_Types.h.

6.1.4.3 Flexray_Ip_PocStateType

```
enum Flexray_Ip_PocStateType
```

Flexray Protocol State.

Enumerator

FLEXRAY_STATE_CONFIG	POC:default config state
FLEXRAY_STATE_DEFAULT_CONFIG	POC:config state
FLEXRAY_STATE_HALT	POC:halt state
FLEXRAY_STATE_NORMAL_ACTIVE	POC:normal active state
FLEXRAY_STATE_NORMAL_PASSIVE	POC:normal passive state
FLEXRAY_STATE_READY	POC:ready state
FLEXRAY_STATE_STARTUP	POC:startup state
FLEXRAY_STATE_WAKEUP	POC:wakeup state

Definition at line 235 of file Flexray_Ip_Types.h.

6.1.4.4 Flexray_Ip_ExtClkCorrectType

```
enum Flexray_Ip_ExtClkCorrectType
```

External Offset Correction Application Using to trigger the application of the external offset correction value defined in the Protocol Configuration.

Enumerator

FLEXRAY_EXT_CLK_NOT_APPLY	do not apply external rate correction value
FLEXRAY_EXT_CLK_SUBTRACT	subtract external correction value
FLEXRAY_EXT_CLK_ADD	add external correction value

Definition at line 254 of file Flexray_Ip_Types.h.

6.1.4.5 Flexray_Ip_GlobalIntType

enum `Flexray_Ip_GlobalIntType`

Global interrupt flags.

Enumerator

<code>FLEXRAY_MODULE_INTERRUPT</code>	set if at least one of the other interrupt flags is set
<code>FLEXRAY_PROTOCOL_INTERRUPT</code>	set if at least one of the individual flags in the Protocol Interrupt Flags is set
<code>FLEXRAY_CHI_INTERRUPT</code>	set if at least one of the error flags in the CHI Error Flag Register is set
<code>FLEXRAY_WAKEUP_INTERRUPT</code>	set when the CC has received a wakeup symbol on the FlexRay bus
<code>FLEXRAY_FIFOA_INTERRUPT</code>	set if FIFO A almost full event has occurred
<code>FLEXRAY_FIFOB_INTERRUPT</code>	set if FIFO B almost full event has occurred

Definition at line 266 of file Flexray_Ip_Types.h.

6.1.4.6 Flexray_Ip_WakeupStatusType

enum `Flexray_Ip_WakeupStatusType`

Flexray Wakeup Status.

Enumerator

<code>FLEXRAY_UNDEFINED</code>	undefined status
<code>FLEXRAY_RECEIVED_HEADER</code>	received header status
<code>FLEXRAY_RECEIVED_WUP</code>	received wakeup status
<code>FLEXRAY_COLLISION_HEADER</code>	collision header status
<code>FLEXRAY_COLLISION_WUP</code>	wakeup collision status
<code>FLEXRAY_COLLISION_UNKNOWN</code>	unknown collision status
<code>FLEXRAY_TRANSMITTED_WUP</code>	transmitted wakeup status

Definition at line 281 of file Flexray_Ip_Types.h.

6.1.4.7 Flexray_Ip_StartupStatusType

enum `Flexray_Ip_StartupStatusType`

Flexray Startup Status, pegged to the AutoSAR enum, but uses SDK element names.

Definition at line 298 of file `Flexray_Ip_Types.h`.

6.1.4.8 Flexray_Ip_ProtocolIntType

enum `Flexray_Ip_ProtocolIntType`

Protocol Interrupt Flags.

This is shift bit value for each protocol interrupt flag when merging two registers: PIFR0 and PIFR1

Enumerator

<code>FLEXRAY_CYCLE_START</code>	set when a communication cycle starts
<code>FLEXRAY_T1_EXPIRED</code>	set whenever timer 1 expires
<code>FLEXRAY_T2_EXPIRED</code>	set whenever timer 2 expires
<code>FLEXRAY_TX_ACROSS_BOUNDARY_A</code>	set when the frame transmission on channel A crosses the slot boundary
<code>FLEXRAY_TX_ACROSS_BOUNDARY_B</code>	set when the frame transmission on channel B crosses the slot boundary
<code>FLEXRAY_PLATESTTX_VIO_A</code>	set when the frame transmission on channel A in the dynamic segment exceeds the dynamic segment boundary.
<code>FLEXRAY_PLATESTTX_VIO_B</code>	set when the frame transmission on channel B in the dynamic segment exceeds the dynamic segment boundary
<code>FLEXRAY_MEDIA_ACCESS_TEST_SYMBOL_← _RECEIVED</code>	set when the MTS symbol was received on channel A or channel B
<code>FLEXRAY_MAX_SYNC_FRAMES</code>	set when more than <code>node_sync_max</code> sync frames detected
<code>FLEXRAY_CLOCK_CORRECTION_LIMIT</code>	set when offset or rate correction limit reached
<code>FLEXRAY_MISSING_OFFSET_CORRECTION</code>	set when an insufficient number of measurements is available for offset correction.
<code>FLEXRAY_MISSING_RATE_CORRECTION</code>	set when an insufficient number of measurements is available for rate correction at the end of the communication cycle.
<code>FLEXRAY_COLD_START_ABORT</code>	set when the configured number of allowed cold start attempts is reached and none of these attempts was successful
<code>FLEXRAY_ILLEGAL_PROTOCOL_CONFIGU← RATION</code>	set when the protocol engine has detected an illegal protocol configuration parameter setting

Enumerator

FLEXRAY_INTERNAL_PROTOCOL_ERR	set when the protocol engine has detected an internal protocol error
FLEXRAY_FATAL_PROTOCOL_ERR	set when the protocol engine has detected a fatal protocol error
FLEXRAY_ODD_CYCLE_TABLE	set if the CC has written the sync frame measurement / ID tables into the FlexRay memory area for the odd cycle.
FLEXRAY_EVEN_CYCLE_TABLE	set if the CC has written the sync frame measurement / ID tables into the FlexRay memory area for the even cycle.
FLEXRAY_SSC0_INC	set when the SLOTSTATUSCNT field in the corresponding Slot Status Counter Registers is incremented
FLEXRAY_SSC1_INC	set when the SLOTSTATUSCNT field in the corresponding Slot Status Counter Registers is incremented
FLEXRAY_SSC2_INC	set when the SLOTSTATUSCNT field in the corresponding Slot Status Counter Registers is incremented
FLEXRAY_SSC3_INC	set when the SLOTSTATUSCNT field in the corresponding Slot Status Counter Registers is incremented
FLEXRAY_PROTOCOL_STATE_CHANGED	set when protocol state changed
FLEXRAY_PE_COMM_FAILURE	set if the CC has detected a communication failure between the PE and the CHI.
FLEXRAY_ILLEGAL_CTRL_CMD	set when illegal protocol control command detected
FLEXRAY_ERROR_MODE_CHANGED	set when ERRMODE field changed

Definition at line 328 of file Flexray_Ip_Types.h.

6.1.4.9 Flexray_Ip_ChiIntType

```
enum Flexray_Ip_ChiIntType
```

CHI Error Flags.

Enumerator

FLEXRAY_FRAME_LOST_B	set if frame lost on channel B detected
FLEXRAY_FRAME_LOST_A	set if frame lost on channel A detected
FLEXRAY_CMD_IGNORED	set if POC command ignored
FLEXRAY_FIFO_OVERRUN_B	set if FIFO overrun on channel B has been detected
FLEXRAY_FIFO_OVERRUN_A	set if FIFO overrun on channel A has been detected
FLEXRAY_MB_SEARCH_ERR	set if search engine active while search start appears or illegal message buffer index detected

Enumerator

FLEXRAY_MB_UTILIZE_ERR	asserted if the application writes to a message buffer control field that is beyond the number of utilized message buffers programmed in the Message Buffer Segment Size and Utilization Register
FLEXRAY_MB_LOCK_ERR	set if lock error detected
FLEXRAY_SYS_BUS_ERR	set if system bus access not finished in time
FLEXRAY_FRAME_ID_ERR	set if frame id error occurred
FLEXRAY_DYNAMIC_PAY_LEN_ERR	set if dynamic payload length error occurred
FLEXRAY_STATIC_PAY_LEN_ERR	set if static payload length error occurred
FLEXRAY_NMV_LEN_ERR	set if network management length error occurred
FLEXRAY_NM_FRAME_ERR	set if network management frame error occurred
FLEXRAY_ILL_SYS_BUS_ADDR	set if the external system bus subsystem has detected an access to an illegal system bus address from the CC

Definition at line 368 of file Flexray_Ip_Types.h.

6.1.4.10 Flexray_Ip_WakeUpChannelType

```
enum Flexray_Ip_WakeUpChannelType
```

Choose wakeup for channel.

Enumerator

FLEXRAY_WUP_NO_WAKEUP	No wake up both channel A & B
FLEXRAY_WUP_A	Wake up signal on channel A
FLEXRAY_WUP_B	Wake up signal on channel B
FLEXRAY_WUP_AB	Wake up signal on channel A & B

Definition at line 395 of file Flexray_Ip_Types.h.

6.1.4.11 Flexray_Ip_MemLayoutType

```
enum Flexray_Ip_MemLayoutType
```

Layout for memory message buffer.

Enumerator

FLEXRAY_MEM_LAYOUT_MIXED	FIFO Base Address located in System Memory Base Address
FLEXRAY_MEM_LAYOUT_DEDICATED	FIFO Base Address located in Receive FIFO System Memory Base Address

Definition at line 407 of file Flexray_Ip_Types.h.

6.1.4.12 Flexray_Ip_RangeFilterModeType

```
enum Flexray_Ip_RangeFilterModeType
```

Enum for range filter type.

Enumerator

FR_ACCEPTANCE	Fifo range filter accept mode
FR_REJECTION	Fifo range filter reject mode

Definition at line 417 of file Flexray_Ip_Types.h.

6.1.4.13 Flexray_Ip_CounterIncType

```
enum Flexray_Ip_CounterIncType
```

Enum for counter increment type.

Enumerator

FLEXRAY_COUNTER_INC_BY_1_A	increment by 1 if condition is fulfilled on channel A
FLEXRAY_COUNTER_INC_BY_1_B	increment by 1 if condition is fulfilled on channel B
FLEXRAY_COUNTER_INC_BY_1_AB	increment by 1 if condition is fulfilled on at least one channel.
FLEXRAY_COUNTER_INC_BY_1_2	increment by 2 if condition is fulfilled on both channels channel; increment by 1 if condition is fulfilled on only one channel.

Definition at line 428 of file Flexray_Ip_Types.h.

6.1.4.14 Flexray_Ip_TimerType

enum `Flexray_Ip_TimerType`

Enum for choose timer.

Definition at line 442 of file `Flexray_Ip_Types.h`.

6.1.4.15 Flexray_Ip_BitRateType

enum `Flexray_Ip_BitRateType`

FlexRay Bus Bit Rate.

Enumerator

<code>FLEXRAY_BITRATE_10</code>	10 Mbit/s
<code>FLEXRAY_BITRATE_5</code>	5 Mbit/s
<code>FLEXRAY_BITRATE_2↔ _5</code>	2.5 Mbit/s
<code>FLEXRAY_BITRATE_8</code>	8.0 Mbit/s

Definition at line 452 of file `Flexray_Ip_Types.h`.

6.1.4.16 Flexray_Ip_SoftwareStatesType

enum `Flexray_Ip_SoftwareStatesType`

FlexRay software states.

Enumerator

<code>FLEXRAY_STATE_UNINIT</code>	Flexray un-init
<code>FLEXRAY_STATE_INIT</code>	Flexray init

Definition at line 464 of file `Flexray_Ip_Types.h`.

6.1.5 Function Reference

6.1.5.1 Flexray_Ip_GetCurrentState()

```
Flexray_Ip_PocStateType Flexray_Ip_GetCurrentState (
    uint8 instNum )
```

Return current protocol state.

Parameters

in	<i>instNum</i>	Flexray instance number
----	----------------	-------------------------

Returns

Current protocol state

6.1.5.2 Flexray_Ip_WaitPOCState()

```
Flexray_Ip_StatusType Flexray_Ip_WaitPOCState (
    uint8 instNum,
    Flexray_Ip_PocStateType pocState )
```

Waits for the POC to change its state.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>pocState</i>	Current state of POC

Returns

FLEXRAY_STATUS_SUCCESS/FLEXRAY_STATUS_TIMEOUT

6.1.5.3 Flexray_Ip_ConfigTimer()

```
void Flexray_Ip_ConfigTimer (
    uint8 instNum,
    const Flexray_Ip_TimerConfigType * timerCfg )
```

Config timer operation.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>timerCfg</i>	Contain all configuration of timer

6.1.5.4 Flexray_Ip_StartTimer()

```
void Flexray_Ip_StartTimer (
    uint8 instNum,
    Flexray_Ip_TimerType timer )
```

Start timer.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>number</i>	Determine which timer will start

6.1.5.5 Flexray_Ip_StopTimer()

```
void Flexray_Ip_StopTimer (
    uint8 instNum,
    Flexray_Ip_TimerType timer )
```

Stop timer.

Parameters

in	<i>number</i>	determine which timer will stop
----	---------------	---------------------------------

6.1.5.6 Flexray_Ip_SetStatusCounter()

```
void Flexray_Ip_SetStatusCounter (
    uint8 instNum,
    uint8 counterIndex,
    const Flexray_Ip_CounterConfigType * counterConfig )
```

Setup status counter.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>counterIndex</i>	Select a status counter. Value must be in range 0-3.
in	<i>counterConfig</i>	Counter configuration.

6.1.5.7 Flexray_Ip_GetStatusCounter()

```
uint16 Flexray_Ip_GetStatusCounter (
    uint8 instNum,
    uint8 counterIndex )
```

Get status counter.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>counterIndex</i>	Select a status counter. Value must be in range 0-3.

Returns

The current value of the Slot Status Counter.

6.1.5.8 Flexray_Ip_SetProtocolInterrupt()

```
void Flexray_Ip_SetProtocolInterrupt (
    uint8 instNum,
    Flexray_Ip_ProtocolIntType pInt,
    boolean enable )
```

Sets the chosen protocol interrupt.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>pInt</i>	The protocol interrupt
in	<i>enable</i>	true if enabled; false if disabled

6.1.5.9 Flexray_Ip_GetProtocolInterruptFlag()

```
boolean Flexray_Ip_GetProtocolInterruptFlag (
    uint8 instNum,
    Flexray_Ip_ProtocolIntType pInt )
```

Gets the chosen protocol interrupt flag value.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>pInt</i>	The protocol interrupt

Returns

true if flag is raised; false if flag isn't raised.

6.1.5.10 Flexray_Ip_ClearProtocolInterruptFlag()

```
void Flexray_Ip_ClearProtocolInterruptFlag (
    uint8 instNum,
    Flexray_Ip_ProtocolIntType pInt )
```

Clears the chosen protocol interrupt flag.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>pInt</i>	The protocol interrupt

6.1.5.11 Flexray_Ip_GetGlobalInterruptFlag()

```
boolean Flexray_Ip_GetGlobalInterruptFlag (
    uint8 instNum,
    Flexray_Ip_GlobalIntType gInt )
```

Gets the chosen global interrupt flag value.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>gInt</i>	The global interrupt

Returns

true if flag is raised; false if flag isn't raised.

6.1.5.12 Flexray_Ip_SetGlobalInterrupt()

```
void Flexray_Ip_SetGlobalInterrupt (
    uint8 instNum,
    Flexray_Ip_GlobalIntType gInt,
    boolean enable )
```

Sets the chosen global interrupt.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>gInt</i>	The global interrupt
in	<i>enable</i>	true if enabled; false if disabled

6.1.5.13 Flexray_Ip_ClearGlobalInterruptFlag()

```
void Flexray_Ip_ClearGlobalInterruptFlag (
    uint8 instNum,
    Flexray_Ip_GlobalIntType gInt )
```

Clears the chosen global interrupt flag.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>gInt</i>	The global interrupt

6.1.5.14 Flexray_Ip_GetCHIInterruptFlag()

```
boolean Flexray_Ip_GetCHIInterruptFlag (
    uint8 instNum,
    Flexray_Ip_ChiIntType cInt )
```

Gets the chosen controller host interface interrupt flag value.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>cInt</i>	The chi interrupt

Returns

true if flag is raised; false if flag isn't raised.

6.1.5.15 Flexray_Ip_ClearCHIInterruptFlag()

```
void Flexray_Ip_ClearCHIInterruptFlag (
    uint8 instNum,
    Flexray_Ip_ChiIntType cInt )
```

Clears the chosen controller host interface interrupt flag.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>cInt</i>	The chi interrupt

6.1.5.16 Flexray_Ip_SetStatusMonitorSlot()

```
void Flexray_Ip_SetStatusMonitorSlot (
    uint8 instNum,
    uint8 monitorIndex,
    uint16 slotNumber )
```

Setup the slot status monitor.

Monitor the most recent status of a slot

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>monitorIndex</i>	Select slot status monitor. Value must be in range 0-3.
in	<i>slotNumber</i>	Select the slot to monitor. See Slot Status Monitoring in RM for details

6.1.5.17 Flexray_Ip_GetStatusMonitor()

```
uint8 Flexray_Ip_GetStatusMonitor (
    uint8 instNum,
    uint8 monitorIndex,
    boolean channelA,
    boolean evenCycle )
```

Get slot status from slot status monitor.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>monitorIndex</i>	Select slot status monitor. Value must be in range 0-3.
in	<i>channelA</i>	True: get slot status from channel A. False: get slot status from channel B
in	<i>evenCycle</i>	True: get slot status from the last even cycle. False: get slot status from the last odd cycle.

Returns

Slot status content. Each bit is a protocol related variable. See FR_SSRn field descriptions table in RM for details.

6.1.5.18 Flexray_Ip_SetExternalCorrection()

```
void Flexray_Ip_SetExternalCorrection (
    uint8 instNum,
    boolean rateCorrect,
    Flexray_Ip_ExtClkCorrectType correctType )
```

Apply external rate or offset correction value.

See External Clock Synchronization section in RM for details.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>rateCorrect</i>	true if correct rate, false if correct offset
in	<i>correctType</i>	Correction type

6.1.5.19 Flexray_Ip_SetMTS()

```
Flexray_Ip_StatusType Flexray_Ip_SetMTS (
    uint8 instNum,
```



```

boolean channelA,
boolean enable )

```

Enables or disables the generation of the MTS on either channel.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>channelA</i>	True: Select channel A. False: channel B.
in	<i>enable</i>	Enable or disable MTS transmission(True: enable/False: disable)

Returns

FLEXRAY_STATUS_SUCCESS: Enable/disable generate MTS successfully FLEXRAY_STATUS_ERR↵
 OR: When the POC state isn't POC:NORMAL ACTIVE state

6.1.5.20 Flexray_Ip_GetCurrentSlotCounter()

```

uint16 Flexray_Ip_GetCurrentSlotCounter (
    uint8 instNum,
    boolean channelA )

```

Get current slot counter on channel.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>channelA</i>	True: Select channel A. False: channel B.

Returns

Current slot counter value

6.1.5.21 Flexray_Ip_Init()

```

Flexray_Ip_StatusType Flexray_Ip_Init (
    uint8 instNum,
    const Flexray_Ip_ConfigType * config,
    const Flexray_Ip_StateAllocateType * stateAlloc )

```

Initialize CC, PE, message buffers and assign user callback.

After this function return, the module will be in POC:ready state. The function will send CONFIG, CONFIG_↵
 COMPLETE command and wait until state changed or time out.

Parameters

in	<i>instNum</i>	Instance number
in	<i>config</i>	User config structure
in	<i>stateAlloc</i>	Pointer to the array of state structure. Array size must be equal the number of message buffers used. This memory area for its internal logic. The application must make no assumptions about the content of this structure, and must not free this memory until the driver is de-initialized.

Returns

FLEXRAY_STATUS_SUCCESS: Initialize successfully FLEXRAY_STATUS_TIMEOUT: Timed out while waiting for state change FLEXRAY_STATUS_FIFO_A_ERROR_CONFIG : FiFo A will operate incorrectly if sum of the current fifo depth value and the previous start index value is smaller or equal than the previous read index on channel A. FLEXRAY_STATUS_FIFO_B_ERROR_CONFIG : FiFo B will operate incorrectly if sum of the current fifo depth value and the previous start index value is smaller or equal than the previous read index on channel B. FLEXRAY_STATUS_FIFO_AB_ERROR_CONFIG : FiFo A & B will operate incorrectly if sum of the current fifo depth value and the previous start index value is smaller or equal than the previous read index on both two channel. FLEXRAY_STATUS_ERROR: When the POC state isn't POC:DEFAULT CONFIG state before initialize or setup rx FiFo unsuccessfully

6.1.5.22 Flexray_Ip_Deinit()

```
Flexray_Ip_StatusType Flexray_Ip_Deinit (
    uint8 instNum )
```

Execute shutdown sequence.

This function execute the shutdown sequence described in RM: Disable all enabled message buffers and issue HALT command. Note that this is blocking function since it poll for enabled/disabled bit in each individual message buffer, and wait for POC:halt state before return.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
----	----------------	--

Returns

FLEXRAY_STATUS_SUCCESS: shutdown successfully FLEXRAY_STATUS_TIMEOUT: Timed out while waiting for disabled status of individual buffer or while waiting for POC:halt

6.1.5.23 Flexray_Ip_ReceiveInd()

```
void Flexray_Ip_ReceiveInd (
    uint8 instNum,
```

```
uint8 bufferId,
Flexray_Ip_FrameHeaderType * frameHdr,
uint8 * pData,
uint8 dataLength )
```

Receive an individual buffer. Return immediately.

If message buffer have new unread content, this function read it immediately and GetTransferStatus shall return SUCCESS. If message buffer have no unread content, this function shall not read the buffer and GetTransferStatus shall return BUSY, transfer operation will success when a new frame is received.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>bufferId</i>	Individual buffer index
out	<i>hdr</i>	Pointer to a frame header structure
out	<i>data</i>	Payload data buffer to receive
in	<i>dataLength</i>	data length of input buffer in 2-byte unit

6.1.5.24 Flexray_Ip_Send()

```
Flexray_Ip_StatusType Flexray_Ip_Send (
    uint8 instNum,
    uint8 bufferId,
    const uint8 * pData,
    uint8 dataLength )
```

Send an individual buffer. Return immediately.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>bufferId</i>	Individual buffer index
in	<i>data</i>	Payload data to transmit
in	<i>dataLength</i>	data length of input buffer in 2-byte unit

6.1.5.25 Flexray_Ip_SendBlocking()

```
Flexray_Ip_StatusType Flexray_Ip_SendBlocking (
    uint8 instNum,
    uint8 bufferId,
    const uint8 * pData,
```

```
uint8 dataLength,
uint32 timeout_ms )
```

Send an individual buffer, wait until the message is sent or time out.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>bufferId</i>	Individual buffer index
in	<i>data</i>	Payload data to transmit
in	<i>dataLength</i>	data length of input buffer in 2-byte unit
in	<i>timeout_ms</i>	timeout value in milliseconds

Returns

FLEXRAY_STATUS_TX_CONFLICT_A: The transfer is completed and transmit conflict on channel A is reported by hardware
 FLEXRAY_STATUS_TX_CONFLICT_B: The transfer is completed and transmit conflict on channel B is reported by hardware
 FLEXRAY_STATUS_TX_CONFLICT_AB: The transfer is completed and transmit conflict on channel A & B is reported by hardware
 FLEXRAY_STATUS_SUCCESS : The transfer is completed and no transmit conflict is reported by hardware.
 FLEXRAY_STATUS_TIMEOUT: Timed out before the transmission complete

6.1.5.26 Flexray_Ip_ReceiveIndBlocking()

```
Flexray_Ip_StatusType Flexray_Ip_ReceiveIndBlocking (
    uint8 instNum,
    uint8 bufferId,
    Flexray_Ip_FrameHeaderType * frameHdr,
    uint8 * pData,
    uint8 dataLength,
    uint32 timeout_ms )
```

Receive an individual buffer. Wait until message is received or time out.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>bufferId</i>	Individual buffer index
out	<i>hdr</i>	Pointer to a frame header structure
out	<i>data</i>	Payload data buffer to receive
in	<i>dataLength</i>	data length of input buffer in 2-byte unit
in	<i>timeout_ms</i>	timeout value in milliseconds

Returns

FLEXRAY_STATUS_SUCCESS: The reception is successful. FLEXRAY_STATUS_TIMEOUT: Timed out before the reception complete

6.1.5.27 Flexray_Ip_ReceiveFIFO()

```
Flexray_Ip_StatusType Flexray_Ip_ReceiveFIFO (
    uint8 instNum,
    boolean channel,
    Flexray_Ip_FrameHeaderType * frameHdr,
    uint8 * pData,
    uint8 dataLength )
```

Receive a buffer from fifo. Return immediately.

Parameters

in	<i>channel</i>	access fifo channel A/B(false/true)
out	<i>frameHdr</i>	pointer to output header structure
out	<i>data</i>	pointer to output buffer data
in	<i>dataLength</i>	data length of input buffer in 2-byte unit

Returns

FLEXRAY_STATUS_SUCCESS: Read from individual buffer successfully. FLEXRAY_STATUS_ERROR: No entry in fifo

6.1.5.28 Flexray_Ip_GetTransferStatus()

```
Flexray_Ip_StatusType Flexray_Ip_GetTransferStatus (
    uint8 instNum,
    uint8 bufferId )
```

Get status of an individual buffer transfer.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>bufferId</i>	Individual buffer index

Returns

operation status: FLEXRAY_STATUS_TX_CONFLICT_A: The transfer is completed and transmit conflict on channel A is reported by hardware FLEXRAY_STATUS_TX_CONFLICT_B: The transfer is completed and transmit conflict on channel B is reported by hardware FLEXRAY_STATUS_TX_CONFLICT_AB: The transfer is completed and transmit conflict on channel A & B is reported by hardware FLEXRAY_STATUS_SUCCESS : The transfer is completed and no transmit conflict is reported by hardware. FLEXRAY_STATUS_BUSY : The transfer is not completed FLEXRAY_STATUS_ABORTED: The transfer is aborted by user FLEXRAY_STATUS_TIMEOUT: A blocking transfer has timed out.

6.1.5.29 Flexray_Ip_Abort()

```
Flexray_Ip_StatusType Flexray_Ip_Abort (
    uint8 instNum,
    uint8 bufferId )
```

Abort transfer of an individual buffer. For Rx, If abort function is successful, memory of this buffer will be not updated about frame header and payload. For Tx, If abort function is successful, tx message buffer will be not sent on bus. If the application want to send/receive message buffer, please call again receive/send function.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>bufferId</i>	Individual buffer index

Returns

FLEXRAY_STATUS_SUCCESS : The transfer is aborted. FLEXRAY_STATUS_ERROR : Cannot abort because the transfer is already completed/ Cannot trigger to lock with txMB/ Cannot trigger to disable rxMB

6.1.5.30 Flexray_Ip_GetStartupStatus()

```
Flexray_Ip_StartupStatusType Flexray_Ip_GetStartupStatus (
    uint8 instNum )
```

Return startup status.

Parameters

in	<i>instNum</i>	Flexray instance number
----	----------------	-------------------------

Returns

Startup status

6.1.5.31 Flexray_Ip_GetCurrentCycleCount()

```
uint8 Flexray_Ip_GetCurrentCycleCount (
    uint8 instNum )
```

Return current cycle count.

Parameters

in	<i>instNum</i>	Flexray instance number
----	----------------	-------------------------

6.1.5.32 Flexray_Ip_GetWakeupChannel()

```
Flexray_Ip_WakeUpChannelType Flexray_Ip_GetWakeupChannel (
    uint8 instNum )
```

Return wakeup channel.

Parameters

in	<i>instNum</i>	Flexray instance number
----	----------------	-------------------------

Returns

Wake up channel

6.1.5.33 Flexray_Ip_GetWakeupStatus()

```
Flexray_Ip_WakeupStatusType Flexray_Ip_GetWakeupStatus (
    uint8 instNum )
```

Return wakeup status.

Parameters

in	<i>instNum</i>	Flexray instance number
----	----------------	-------------------------

Returns

Wake up status

6.1.5.34 Flexray_Ip_GetDefaultConfig()

```
void Flexray_Ip_GetDefaultConfig (
    const Flexray_Ip_ConfigType * config )
```

Get PE and CC default configuration.

Parameters

out	<i>config</i>	pointer to user configuration.
-----	---------------	--------------------------------

6.1.5.35 Flexray_Ip_SetBasicDynamicTxFrame()

```
void Flexray_Ip_SetBasicDynamicTxFrame (
    uint8 instNum,
    uint8 bufferId,
    boolean PPI,
    uint8 payloadLength )
```

Set PPI or payloadLength for Dynamic frame in individual buffer failed.

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>MBIndex</i>	Index of individual message buffer
in	<i>PPI</i>	new PPI of transmitted dynamic frame
in	<i>payloadLength(unit</i>	is 2 bytes): new payloadLength of transmitted dynamic frame.

6.1.5.36 Flexray_Ip_ReConfigIndividual()

```
Flexray_Ip_StatusType Flexray_Ip_ReConfigIndividual (
```



```
uint8 instNum,
Flexray_Ip_IndConfigType * pIndConfig )
```

Reconfig an individual buffer. Return immediately. The field phyMBIndex in struct [Flexray_Ip_IndConfigType](#) cannot be reconfigured. The field MBIndex in struct [Flexray_Ip_IndConfigType](#) cannot be reconfigured.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
in	<i>pIndConfig</i>	Individual buffer configuration structure

Returns

FLEXRAY_STATUS_SUCCESS: Setup successfully. FLEXRAY_STATUS_ERROR: the trigger to disable/enable individual buffer failed.

6.1.5.37 Flexray_Ip_GetNMVector()

```
Flexray_Ip_StatusType Flexray_Ip_GetNMVector (
    uint8 instNum,
    uint8 * data,
    uint8 * configLen )
```

Return network management vector of last cycle.

Parameters

in	<i>instNum</i>	FLEXRAY Hardware Interface instance number
out	<i>data</i>	Memory to store network management vector data
in	<i>length</i>	Size in byte of memory provided by data

Returns

Length of network management vector configured from POC:config

6.1.5.38 Flexray_Ip_GetFifoLevel()

```
uint8 Flexray_Ip_GetFifoLevel (
    uint8 instNum,
    boolean isChnB )
```

Return number of entries in fifo.

Module Documentation

Parameters

in	<i>instNum</i>	Flexray instance number
in	<i>isChnB</i>	True: Fifo channel B. False: Fifo channel A.

Returns

Number of entries.

6.1.5.39 FLEXRAY_SetFiFoRangeFilter()

```
void FLEXRAY_SetFiFoRangeFilter (
    FLEXRAY_Type * base,
    uint8 filterNum,
    const Flexray_Ip_RangeFilterType * filterCfg )
```

Set FiFo Range Filter.

Parameters

in	<i>base</i>	The Flexray base address
in	<i>filterNum</i>	Number of filter
in	<i>filterCfg</i>	Point to range filter configuration structure (Flexray_Ip_RangeFilterType)

6.1.5.40 FLEXRAY_InitPE()

```
void FLEXRAY_InitPE (
    uint8 instNum,
    const Flexray_Ip_PeConfigType * peConfig )
```

Set the protocol engine configuration.

Parameters

in	<i>base</i>	The Flexray base address
in	<i>peConfig</i>	point to protocol engine configuration structure (Flexray_Ip_PeConfigType)

6.1.5.41 FLEXRAY_ConfigureTimer1()

```
void FLEXRAY_ConfigureTimer1 (
    FLEXRAY_Type * base,
    const Flexray_Ip_TimerConfigType * timerCfg )
```

Configure timer 1.

Parameters

in	<i>base</i>	The Flexray base address
in	<i>timeCfg</i>	Pointer to timer configuration structure

6.1.5.42 FLEXRAY_ConfigureTimer2()

```
void FLEXRAY_ConfigureTimer2 (
    FLEXRAY_Type * base,
    const Flexray_Ip_TimerConfigType * timerCfg )
```

Configure timer 2.

Parameters

in	<i>base</i>	The Flexray base address
in	<i>timeCfg</i>	Pointer to timer configuration structure

6.2 FR_DRIVER

6.2.1 Detailed Description

Macros

- `#define FR_E_INV_TIMER_IDX`
Error reported when parameter timer index exceeds number of available timers.
- `#define FR_E_PARAM_POINTER`
Error reported when invalid pointer in parameter list.
- `#define FR_E_INV_OFFSET`
Error reported when parameter offset exceeds bounds.
- `#define FR_E_INV_CTRL_IDX`
Error reported when invalid controller index.
- `#define FR_E_INV_CHNL_IDX`
Error reported when invalid channel index.
- `#define FR_E_INV_CYCLE`
Error reported when parameter cycle exceeds 63.
- `#define FR_E_INIT_FAILED`
Error reported when Fr module was not initialized.
- `#define FR_E_INV_POCSTATE`
Error reported when Fr CC is not in the expected POC state.
- `#define FR_E_INV_LENGTH`
Error reported when Payload length parameter has an invalid value.
- `#define FR_E_INV_LPDU_IDX`
Error reported when invalid LPdu index.
- `#define FR_E_INV_HEADERCRC`
Error reported when invalid FlexRay header CRC.
- `#define FR_E_INV_CONFIG_IDX`
Error reported when invalid value passed as parameter Fr_ConfigParamIdx.
- `#define FR_E_INV_FRAMELIST_SIZE`
Error reported when invalid framelist size value.
- `#define FR_E_PARAM_CONFIG`
Error reported when requested resource is not configured to be available on the current core.

Function Reference

- void `Fr_GetVersionInfo` (Std_VersionInfoType *VersioninfoPtr)
Software module version query.
- void `Fr_Init` (const Fr_ConfigType *Fr_ConfigPtr)
Initializes the Fr.
- Std_ReturnType `Fr_ControllerInit` (uint8 Fr_CtrlIdx)
CC configuration.
- Std_ReturnType `Fr_SetWakeupChannel` (uint8 Fr_CtrlIdx, Fr_ChannelType Fr_ChnlIdx)

- Selects which channel sends WUPs.*
- Std_ReturnType [Fr_CancelTxLPdu](#) (uint8 Fr_CtrlIdx, uint16 Fr_LPduIdx)
 - Cancels the already pending transmission of a LPdu contained in a controllers physical transmit resource.*
- Std_ReturnType [Fr_ReceiveRxLPdu](#) (uint8 Fr_CtrlIdx, uint16 Fr_LPduIdx, uint8 *Fr_LSduPtr, Fr_↵ RxLPduStatusType *Fr_LPduStatusPtr, uint8 *Fr_LSduLengthPtr, Fr_SlotAssignmentType *Fr_Slot↵ AssignmentPtr)
 - Receives data from the FlexRay network.*
- Std_ReturnType [Fr_TransmitTxLPdu](#) (uint8 Fr_CtrlIdx, uint16 Fr_LPduIdx, const uint8 *Fr_LSduPtr, uint8 Fr_LSduLength, Fr_SlotAssignmentType *Fr_SlotAssignmentPtr)
 - Transmits data on the FlexRay network.*
- Std_ReturnType [Fr_SendWUP](#) (uint8 Fr_CtrlIdx)
 - Invokes the CC CHI command WAKEUP.*
- Std_ReturnType [Fr_GetNmVector](#) (uint8 Fr_CtrlIdx, uint8 *Fr_NmVectorPtr)
 - Gets the network management vector of the last communication cycle.*
- Std_ReturnType [Fr_AbortCommunication](#) (uint8 Fr_CtrlIdx)
 - Invokes the CC CHI command FREEZE.*
- Std_ReturnType [Fr_AllowColdstart](#) (uint8 Fr_CtrlIdx)
 - Invokes the CC CHI command ALLOW_COLDSTART.*
- Std_ReturnType [Fr_HaltCommunication](#) (uint8 Fr_CtrlIdx)
 - Invokes the CC CHI command DEFERRED_HALT.*
- Std_ReturnType [Fr_StartCommunication](#) (uint8 Fr_CtrlIdx)
 - Starts communication.*
- Std_ReturnType [Fr_AllSlots](#) (uint8 Fr_CtrlIdx)
 - Invokes the CC CHI command ALL_SLOTS.*
- Std_ReturnType [Fr_CheckTxLPduStatus](#) (uint8 Fr_CtrlIdx, uint16 Fr_LPduIdx, Fr_TxLPduStatusType *Fr_TxLPduStatusPtr, Fr_SlotAssignmentType *Fr_SlotAssignmentPtr)
 - Checks the transmit status of the LPdu.*
- Std_ReturnType [Fr_GetPOCStatus](#) (uint8 Fr_CtrlIdx, Fr_POCStatusType *Fr_POCStatusPtr)
 - Gets the POC status.*
- Std_ReturnType [Fr_GetSyncFrameList](#) (uint8 Fr_CtrlIdx, uint8 Fr_ListSize, uint16 *Fr_ChannelAEven↵ ListPtr, uint16 *Fr_ChannelBEvenListPtr, uint16 *Fr_ChannelAOddListPtr, uint16 *Fr_ChannelBOdd↵ ListPtr)
 - Gets a list of syncframes received or transmitted on channel A and channel B via the even and odd communication cycle.*
- Std_ReturnType [Fr_GetChannelStatus](#) (uint8 Fr_CtrlIdx, uint16 *Fr_ChannelAStatusPtr, uint16 *Fr_↵ ChannelBStatusPtr)
 - Gets the channel status information.*
- Std_ReturnType [Fr_GetWakeupRxStatus](#) (uint8 Fr_CtrlIdx, uint8 *Fr_WakeupRxStatusPtr)
 - Gets the wakeup received information from the FlexRay controller.*
- Std_ReturnType [Fr_GetGlobalTime](#) (uint8 Fr_CtrlIdx, uint8 *Fr_CyclePtr, uint16 *Fr_MacroTickPtr)
 - Gets the current global FlexRay time.*
- Std_ReturnType [Fr_GetClockCorrection](#) (uint8 Fr_CtrlIdx, sint16 *Fr_RateCorrectionPtr, sint32 *Fr_↵ OffsetCorrectionPtr)
 - Gets the current clock correction values.*
- Std_ReturnType [Fr_SetAbsoluteTimer](#) (uint8 Fr_CtrlIdx, uint8 Fr_AbsTimerIdx, uint8 Fr_Cycle, uint16 Fr_Offset)
 - Sets the absolute FlexRay timer.*
- Std_ReturnType [Fr_CancelAbsoluteTimer](#) (uint8 Fr_CtrlIdx, uint8 Fr_AbsTimerIdx)

- Stops an absolute timer.*
- Std_ReturnType [Fr_EnableAbsoluteTimerIRQ](#) (uint8 Fr_CtrlIdx, uint8 Fr_AbsTimerIdx)
Enables the interrupt line of an absolute timer.
- Std_ReturnType [Fr_AckAbsoluteTimerIRQ](#) (uint8 Fr_CtrlIdx, uint8 Fr_AbsTimerIdx)
Resets the interrupt condition of an absolute timer.
- Std_ReturnType [Fr_DisableAbsoluteTimerIRQ](#) (uint8 Fr_CtrlIdx, uint8 Fr_AbsTimerIdx)
Disables the interrupt line of an absolute timer.
- Std_ReturnType [Fr_GetAbsoluteTimerIRQStatus](#) (uint8 Fr_CtrlIdx, uint8 Fr_AbsTimerIdx, boolean *Fr_IRQStatusPtr)
Gets IRQ status of an absolute timer.
- Std_ReturnType [Fr_ReadCCCConfig](#) (uint8 Fr_CtrlIdx, uint8 Fr_ConfigParamIdx, uint32 *Fr_ConfigParamValuePtr)
Reads a FlexRay protocol configuration parameter for a particular FlexRay controller out of the module's configuration.
- Std_ReturnType [Fr_GetNumOfStartupFrames](#) (uint8 Fr_CtrlIdx, uint8 *Fr_NumOfStartupFramesPtr)
Gets the current number of startup frames seen on the cluster.

6.2.2 Macro Definition Documentation

6.2.2.1 FR_E_INV_TIMER_IDX

```
#define FR_E_INV_TIMER_IDX
```

Error reported when parameter timer index exceeds number of available timers.

Definition at line 117 of file Fr.h.

6.2.2.2 FR_E_PARAM_POINTER

```
#define FR_E_PARAM_POINTER
```

Error reported when invalid pointer in parameter list.

Definition at line 122 of file Fr.h.

6.2.2.3 FR_E_INV_OFFSET

```
#define FR_E_INV_OFFSET
```

Error reported when parameter offset exceeds bounds.

Definition at line 127 of file Fr.h.

6.2.2.4 FR_E_INV_CTRL_IDX

```
#define FR_E_INV_CTRL_IDX
```

Error reported when invalid controller index.

Definition at line 132 of file Fr.h.

6.2.2.5 FR_E_INV_CHNL_IDX

```
#define FR_E_INV_CHNL_IDX
```

Error reported when invalid channel index.

Definition at line 137 of file Fr.h.

6.2.2.6 FR_E_INV_CYCLE

```
#define FR_E_INV_CYCLE
```

Error reported when parameter cycle exceeds 63.

Definition at line 142 of file Fr.h.

6.2.2.7 FR_E_INIT_FAILED

```
#define FR_E_INIT_FAILED
```

Error reported when Fr module was not initialized.

Definition at line 147 of file Fr.h.

6.2.2.8 FR_E_INV_POCSTATE

```
#define FR_E_INV_POCSTATE
```

Error reported when Fr CC is not in the expected POC state.

Definition at line 152 of file Fr.h.

6.2.2.9 FR_E_INV_LENGTH

```
#define FR_E_INV_LENGTH
```

Error reported when Payload length parameter has an invalid value.

Definition at line 157 of file Fr.h.

6.2.2.10 FR_E_INV_LPDU_IDX

```
#define FR_E_INV_LPDU_IDX
```

Error reported when invalid LPdu index.

Definition at line 162 of file Fr.h.

6.2.2.11 FR_E_INV_HEADERCRC

```
#define FR_E_INV_HEADERCRC
```

Error reported when invalid FlexRay header CRC.

Definition at line 167 of file Fr.h.

6.2.2.12 FR_E_INV_CONFIG_IDX

```
#define FR_E_INV_CONFIG_IDX
```

Error reported when invalid value passed as parameter Fr_ConfigParamIdx.

Definition at line 172 of file Fr.h.

6.2.2.13 FR_E_INV_FRAMELIST_SIZE

```
#define FR_E_INV_FRAMELIST_SIZE
```

Error reported when invalid framelist size value.

Definition at line 177 of file Fr.h.

6.2.2.14 FR_E_PARAM_CONFIG

```
#define FR_E_PARAM_CONFIG
```

Error reported when requested resource is not configured to be available on the current core.

Definition at line 183 of file Fr.h.

6.2.3 Function Reference

6.2.3.1 Fr_GetVersionInfo()

```
void Fr_GetVersionInfo (
    Std_VersionInfoType * VersioninfoPtr )
```

Software module version query.

[SWS_Fr_00341]: The function Fr_GetVersionInfo shall return the version information of this module

Parameters

out	<i>VersioninfoPtr</i>	Std_VersionInfoType type pointer where to store version numbers
-----	-----------------------	---

Returns

void.

6.2.3.2 Fr_Init()

```
void Fr_Init (
    const Fr_ConfigType * Fr_ConfigPtr )
```

Initializes the Fr.

[SWS_Fr_00137]: The function Fr_Init shall internally store the configuration address to enable subsequent API calls to access the configuration.

Parameters

in	<i>Fr_ConfigPtr</i>	Pointer to the Controller configuration
----	---------------------	---

Returns

void.

6.2.3.3 Fr_ControllerInit()

```
Std_ReturnType Fr_ControllerInit (
    uint8 Fr_CtrlIdx )
```

CC configuration.

[SWS_Fr_00148]: Switch CC into POC:Config, configures all FlexRay cluster and node configuration parameters, configures all transmit or receive resources and switch CC into POC:Ready

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
----	-------------------	--

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.4 Fr_SetWakeupChannel()

```
Std_ReturnType Fr_SetWakeupChannel (
    uint8 Fr_CtrlIdx,
    Fr_ChannelType Fr_ChnlIdx )
```

Selects which channel sends WUPs.

[SWS_Fr_00202]: Change the CCs POCState to POC:Config by invoking the CHI command CONFIG, configure the wakeup channel according to parameter Fr_ChnlIdx and change the CCs POCState to POC:Ready again by invoking the CHI command CONFIG_COMPLETE

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_ChnlIdx</i>	Index of FlexRay channel within the context of the FlexRay CC Fr_CtrlIdx

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.5 Fr_CancelTxLPdu()

```
Std_ReturnType Fr_CancelTxLPdu (
    uint8 Fr_CtrlIdx,
    uint16 Fr_LPduIdx )
```

Cancels the already pending transmission of a LPdu contained in a controllers physical transmit resource.

[SWS_Fr_00611]: Figure out the physical resource (e.g., a buffer) mapped to the transmission of the FlexRay frame identified by Fr_LPduIdx. If the physical resource figured out is actively pending for transmission, then the transmit request of this particular resource shall be terminated and E_OK returned. If no transmission is pending E_NOT_OK shall be returned, indicating that no such cancelation took place.

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_LPduIdx</i>	This index is used to uniquely identify a FlexRay frame

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.6 Fr_ReceiveRxLPdu()

```
Std_ReturnType Fr_ReceiveRxLPdu (
    uint8 Fr_CtrlIdx,
    uint16 Fr_LPduIdx,
    uint8 * Fr_LSduPtr,
    Fr_RxLPduStatusType * Fr_LPduStatusPtr,
    uint8 * Fr_LSduLengthPtr,
    Fr_SlotAssignmentType * Fr_SlotAssignmentPtr )
```

Receives data from the FlexRay network.

[SWS_Fr_00233]: This function figures out the physical resource mapped to the reception of the FlexRay frame identified by `Fr_LPduIdx`, figures out whether a new FlexRay frame has been received. If a new frame has been received, copies the received payload data to address `Fr_LSduPtr`, stores the number of bytes and the status `#FR_RECEIVED`. If a new frame has been received, FIFO code is enabled and `Fr_LPduIdx` is associated with FIFO A or FIFO B storage, the received payload data are copied to address `Fr_LSduPtr`, the number of bytes is stored and the status `#FR_RECEIVED` is returned. If no new frame has been received, the function doesn't copy any payload data to `Fr_LSduPtr`, writes 0 to parameter `Fr_LSduLengthPtr` and stores the status `#FR_NOT_RECEIVED`.

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_LPduIdx</i>	This index is used to uniquely identify a FlexRay frame
out	<i>Fr_LSduPtr</i>	Pointer to a buffer where the LSdu
out	<i>Fr_LPduStatusPtr</i>	Pointer to the memory location where the status of the LPdu shall be stored
out	<i>Fr_LSduLengthPtr</i>	Pointer to the memory location where the length of the LSdu (in bytes) shall be stored
out	<i>Fr_SlotAssignmentPtr</i>	This reference points to the memory location where the actual cycle, slot ID, and channel of the frame identified by <code>Fr_LPduIdx</code> shall be stored. A <code>NULL_PTR</code> indicates that the information is not required by the caller.

Returns

`Std_ReturnType`

- `E_OK` API call finished successfully
- `E_NOT_OK` API call aborted due to errors

6.2.3.7 Fr_TransmitTxLPdu()

```
Std_ReturnType Fr_TransmitTxLPdu (
    uint8 Fr_CtrlIdx,
    uint16 Fr_LPduIdx,
    const uint8 * Fr_LSduPtr,
    uint8 Fr_LSduLength,
    Fr_SlotAssignmentType * Fr_SlotAssignmentPtr )
```

Transmits data on the FlexRay network.

[SWS_Fr_00224]: Figures out the physical resource mapped to the transmission of the FlexRay frame identified by `Fr_LPduIdx`, copies `Fr_LSduLength` bytes from address `Fr_LSduPtr` into the FlexRay CCs transmission resource and activate it for transmission

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_LPduIdx</i>	The index is used to uniquely identify a FlexRay frame
in	<i>Fr_LSduPtr</i>	Pointer to a buffer where the assembled LSdu to be transmitted within this LPdu is stored at
in	<i>Fr_LSduLength</i>	Determines the length of the data (in Bytes) to be transmitted
out	<i>Fr_SlotAssignmentPtr</i>	This reference points to the memory location where the actual cycle, slot ID, and channel of the frame identified by <code>Fr_LPduIdx</code> shall be stored. A <code>NULL_PTR</code> indicates that the information is not required by the caller.

Returns

::Std_ReturnType

- E_OK API call has been successful
- E_NOT_OK API call aborted due to errors

6.2.3.8 Fr_SendWUP()

```
Std_ReturnType Fr_SendWUP (
    uint8 Fr_CtrlIdx )
```

Invokes the CC CHI command WAKEUP.

[SWS_Fr_00196]: Invoke the CC CHI command WAKEUP, which initiates the Wakeup Symbol transmission procedure on the configured FlexRay channel

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
----	-------------------	--

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.9 Fr_GetNmVector()

```
Std_ReturnType Fr_GetNmVector (
    uint8 Fr_CtrlIdx,
    uint8 * Fr_NmVectorPtr )
```

Gets the network management vector of the last communication cycle.

[SWS_Fr_00262]: Read the network management vector of the last communication cycle and write it to the output parameter *Fr_NmVectorPtr*. The number of bytes written to the output parameter is constant and known at configuration time

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
out	<i>Fr_NmVectorPtr</i>	Address where the NmVector of the last communication cycle shall be stored

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.10 Fr_AbortCommunication()

```
Std_ReturnType Fr_AbortCommunication (
    uint8 Fr_CtrlIdx )
```

Invokes the CC CHI command FREEZE.

[SWS_Fr_00191]: Invoke the CHI command FREEZE, which immediately aborts the communication and force the CC to the POC:Halt

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
----	-------------------	--

Returns

Std_ReturnType

- E_OK API call has been successful
- E_NOT_OK API call aborted due to errors

6.2.3.11 Fr-AllowColdstart()

```
Std_ReturnType Fr-AllowColdstart (
    uint8 Fr_CtrlIdx )
```

Invokes the CC CHI command ALLOW_COLDSTART.

[SWS_Fr_00182]: Invokes the CC CHI command ALLOW_COLDSTART

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
----	-------------------	--

Returns

Std_ReturnType

- E_OK API call has been successful
- E_NOT_OK API call in bad POC state or failure in access to the controller

6.2.3.12 Fr_HaltCommunication()

```
Std_ReturnType Fr_HaltCommunication (
    uint8 Fr_CtrlIdx )
```

Invokes the CC CHI command DEFERRED_HALT.

[SWS_Fr_00187]: Invokes the CC CHI command HALT, which requests to halt the communication at the end of the current FlexRay communication cycle

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
----	-------------------	--

Returns

Std_ReturnType

- E_OK API call has been successful
- E_NOT_OK API call in bad POC state or failure in access to the controller

6.2.3.13 Fr_StartCommunication()

```
Std_ReturnType Fr_StartCommunication (
    uint8 Fr_CtrlIdx )
```

Starts communication.

[SWS_Fr_00177]: Invoke the CC CHI command RUN, which initiates the startup procedure within the FlexRay CC

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
----	-------------------	--

Module Documentation

Returns

Std_ReturnType

- E_OK API call has been successful
- E_NOT_OK API call aborted due to errors

6.2.3.14 Fr_AllSlots()

```
Std_ReturnType Fr_AllSlots (
    uint8 Fr_CtrlIdx )
```

Invokes the CC CHI command ALL_SLOTS.

[SWS_Fr_00518]: Invoke the CC CHI command ALL_SLOTS, which requests a switch from key slot only mode to all slots transmission mode at the beginning of the next communication cycle.

Parameters

in	Fr_CtrlIdx	Index of FlexRay CC within the context of the FlexRay Driver
----	------------	--

Returns

Std_ReturnType

- E_OK API call has been successful
- E_NOT_OK API call in bad POC state or failure in access to the controller

6.2.3.15 Fr_CheckTxLPduStatus()

```
Std_ReturnType Fr_CheckTxLPduStatus (
    uint8 Fr_CtrlIdx,
    uint16 Fr_LPduIdx,
    Fr_TxLPduStatusType * Fr_TxLPduStatusPtr,
    Fr_SlotAssignmentType * Fr_SlotAssignmentPtr )
```

Checks the transmit status of the LPdu.

[SWS_Fr_00244]: Figures out the physical resource mapped to the transmission of the FlexRay frame identified by Fr_LPduIdx and check whether the transmission resource is still pending for transmission or not

Parameters

in	Fr_CtrlIdx	Index of FlexRay CC within the context of the FlexRay Driver
in	Fr_LPduIdx	This index is used to uniquely identify a FlexRay frame
124 out	Fr_TxLPduStatusPtr	Pointer used to store the transmit status of the LSdu
out	Fr_SlotAssignmentPtr	Pointer used to store cycle, slot ID, and channel of the frame identified by Fr_LPduIdx

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.16 Fr_GetPOCStatus()

```
Std_ReturnType Fr_GetPOCStatus (
    uint8 Fr_CtrlIdx,
    Fr_POCTestStatusType * Fr_POCTestStatusPtr )
```

Gets the POC status.

[SWS_Fr_00217]: Query the CCs actual POC status by reading the CHI variable vPOC and write the result to parameter Fr_POCTestStatusPtr.

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
out	<i>Fr_POCTestStatusPtr</i>	Address the output value is stored to

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.17 Fr_GetSyncFrameList()

```
Std_ReturnType Fr_GetSyncFrameList (
    uint8 Fr_CtrlIdx,
    uint8 Fr_ListSize,
    uint16 * Fr_ChannelAEvenListPtr,
    uint16 * Fr_ChannelBEvenListPtr,
    uint16 * Fr_ChannelAOddListPtr,
    uint16 * Fr_ChannelBOddListPtr )
```

Gets a list of syncframes received or transmitted on channel A and channel B via the even and odd communication cycle.

[SWS_Fr_00575]: Read the list of syncframes received in the last even communication cycle on channel A and write it as array to the memory location `Fr_ChannelAEvenListPtr`. Read the list of syncframes received in the last even communication cycle on channel B and write it as array to the memory location `Fr_ChannelBEvenListPtr`. Read the list of syncframes received in the last odd communication cycle on channel A and write it as array to the memory location `Fr_ChannelAOddListPtr`. Read the list of syncframes received in the last odd communication cycle on channel B and write it as array to the memory location `Fr_ChannelBOddListPtr`.

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver.
in	<i>Fr_ListSize</i>	Size of the arrays passed via parameters: <code>Fr_ChannelAEvenListPtr</code> , <code>Fr_ChannelBEvenListPtr</code> , <code>Fr_ChannelAOddListPtr</code> , <code>Fr_ChannelBOddListPtr</code> .
out	<i>Fr_ChannelAEvenListPtr</i>	Address the list of syncframes on channel A within the even communication cycle is written to.
out	<i>Fr_ChannelBEvenListPtr</i>	Address the list of syncframes on channel B within the even communication cycle is written to.
out	<i>Fr_ChannelAOddListPtr</i>	Address the list of syncframes on channel A within the odd communication cycle is written to.
out	<i>Fr_ChannelBOddListPtr</i>	Address the list of syncframes on channel B within the odd communication cycle is written to.

Returns

`Std_ReturnType`

- `E_OK` API call finished successfully
- `E_NOT_OK` API call aborted due to errors or `Fr_ListSize=0`

6.2.3.18 Fr_GetChannelStatus()

```
Std_ReturnType Fr_GetChannelStatus (
    uint8 Fr_CtrlIdx,
    uint16 * Fr_ChannelAStatusPtr,
    uint16 * Fr_ChannelBStatusPtr )
```

Gets the channel status information.

[SWS_Fr_00558]: Read the aggregated channel status, NIT status, symbol window status and write it to the output parameter `Fr_ChannelAStatusPtr`, `Fr_ChannelBStatusPtr`.

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
out	<i>Fr_ChannelAStatusPtr</i>	Address where the bitcoded channel A status information shall be stored.
out	<i>Fr_ChannelBStatusPtr</i>	Address where the bitcoded channel B status information shall be stored.

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.19 Fr_GetWakeupRxStatus()

```
Std_ReturnType Fr_GetWakeupRxStatus (
    uint8 Fr_CtrlIdx,
    uint8 * Fr_WakeupRxStatusPtr )
```

Gets the wakeup received information from the FlexRay controller.

[SWS_Fr_00588]: Read the wakeup pattern received indicators for channel A and channel B and write it to the output parameter Fr_WakeupRxStatusPtr.

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver.
out	<i>Fr_WakeupRxStatusPtr</i>	Address where bitcoded wakeup reception status shall be stored.

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.20 Fr_GetGlobalTime()

```
Std_ReturnType Fr_GetGlobalTime (
    uint8 Fr_CtrlIdx,
    uint8 * Fr_CyclePtr,
    uint16 * Fr_MacroTickPtr )
```

Gets the current global FlexRay time.

[SWS_Fr_00256]: Read the current global FlexRay time and writes it to the output parameters Fr_cyclePtr and Fr_MacrotickPtr

Module Documentation

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
out	<i>Fr_CyclePtr</i>	Address where the current FlexRay communication cycle value shall be stored
out	<i>Fr_MacroTickPtr</i>	Address where the current macrotick value shall be stored

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.21 Fr_GetClockCorrection()

```
Std_ReturnType Fr_GetClockCorrection (
    uint8 Fr_CtrlIdx,
    sint16 * Fr_RateCorrectionPtr,
    sint32 * Fr_OffsetCorrectionPtr )
```

Gets the current clock correction values.

[SWS_Fr_00566]: Read the rate correction value (vInterimRateCorrection11) and write it as signed integer to the output parameter *Fr_RateCorrectionPtr*. Read the offset correction value (vInterimOffsetCorrection12) and write it as signed integer to the output parameter *Fr_OffsetCorrectionPtr*

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
out	<i>Fr_RateCorrectionPtr</i>	Address where the current rate correction value shall be stored.
out	<i>Fr_OffsetCorrectionPtr</i>	Address where the current offset correction value shall be stored.

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.22 Fr_SetAbsoluteTimer()

```
Std_ReturnType Fr_SetAbsoluteTimer (
    uint8 Fr_CtrlIdx,
    uint8 Fr_AbsTimerIdx,
    uint8 Fr_Cycle,
    uint16 Fr_Offset )
```

Sets the absolute FlexRay timer.

[SWS_Fr_00273]: Program the absolute FlexRay timer `Fr_AbsTimerIdx` according to the parameters `Fr_Cycle` and `Fr_Offset`

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_AbsTimerIdx</i>	Index of absolute timer within the context of the FlexRay CC
in	<i>Fr_Cycle</i>	Absolute cycle the timer shall elapse in
in	<i>Fr_Offset</i>	Offset within cycle <code>Fr_Cycle</code> in units of macrotick the timer shall elapse at

Returns

Std_ReturnType

- `E_OK` API call finished successfully
- `E_NOT_OK` API call aborted due to errors

6.2.3.23 Fr_CancelAbsoluteTimer()

```
Std_ReturnType Fr_CancelAbsoluteTimer (
    uint8 Fr_CtrlIdx,
    uint8 Fr_AbsTimerIdx )
```

Stops an absolute timer.

[SWS_Fr_00287]: Stop the absolute timer `Fr_AbsTimerIdx`

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_AbsTimerIdx</i>	Index of absolute timer within the context of the FlexRay CC

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.24 Fr_EnableAbsoluteTimerIRQ()

```
Std_ReturnType Fr_EnableAbsoluteTimerIRQ (
    uint8 Fr_CtrlIdx,
    uint8 Fr_AbsTimerIdx )
```

Enables the interrupt line of an absolute timer.

[SWS_Fr_00298]: Enable the interrupt line related to timer Fr_AbsTimerIdx

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_AbsTimerIdx</i>	Index of absolute timer within the context of the FlexRay C

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.25 Fr_AckAbsoluteTimerIRQ()

```
Std_ReturnType Fr_AckAbsoluteTimerIRQ (
    uint8 Fr_CtrlIdx,
    uint8 Fr_AbsTimerIdx )
```

Resets the interrupt condition of an absolute timer.

[SWS_Fr_00309]: Reset the interrupt condition of absolute timer Fr_AbsTimerIdx

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_AbsTimerIdx</i>	Index of absolute timer within the context of the FlexRay CC

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.26 Fr_DisableAbsoluteTimerIRQ()

```
Std_ReturnType Fr_DisableAbsoluteTimerIRQ (
    uint8 Fr_CtrlIdx,
    uint8 Fr_AbsTimerIdx )
```

Disables the interrupt line of an absolute timer.

[SWS_Fr_00320]: Disable the interrupt line related to absolute timer Fr_AbsTimerIdx

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_AbsTimerIdx</i>	Index of absolute timer within the context of the FlexRay CC

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.27 Fr_GetAbsoluteTimerIRQStatus()

```
Std_ReturnType Fr_GetAbsoluteTimerIRQStatus (
    uint8 Fr_CtrlIdx,
    uint8 Fr_AbsTimerIdx,
    boolean * Fr_IRQStatusPtr )
```

Gets IRQ status of an absolute timer.

[SWS_Fr_00332]: Check whether the interrupt of absolute timer Fr_AbsTimerIdx is pending. Writes (boolean)T↔RUE to output parameter Fr_IRQStatusPtr in case the interrupt is pending, FALSE otherwise

Module Documentation

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_AbsTimerIdx</i>	Index of absolute timer within the context of the FlexRay CC
out	<i>Fr_IRQStatusPtr</i>	Address the output value is stored to

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.28 Fr_ReadCCConfig()

```
Std_ReturnType Fr_ReadCCConfig (
    uint8 Fr_CtrlIdx,
    uint8 Fr_ConfigParamIdx,
    uint32 * Fr_ConfigParamValuePtr )
```

Reads a FlexRay protocol configuration parameter for a particular FlexRay controller out of the module's configuration.

Read the value of the configuration parameter requested by *Fr_ConfigParamIdx* from the configuration and write it to output parameter **Fr_ConfigParamValuePtr*.

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_ConfigParamIdx</i>	Index that identifies the configuration parameter to read. See macros <code>FR_CIDX_<config_parameter_name></code> .
out	<i>Fr_ConfigParamValuePtr</i>	Address the output value is stored to.

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

6.2.3.29 Fr_GetNumOfStartupFrames()

```
Std_ReturnType Fr_GetNumOfStartupFrames (
    uint8 Fr_CtrlIdx,
    uint8 * Fr_NumOfStartupFramesPtr )
```

Gets the current number of startup frames seen on the cluster.

[SWS_Fr_00549]: Read the number of aligned startup frame pairs received or transmitted during the previous double cycle, aggregated across both channels and write it to the output parameter Fr_NumOfStartupFramesPtr.

Parameters

in	<i>Fr_CtrlIdx</i>	Index of FlexRay CC within the context of the FlexRay Driver
in	<i>Fr_NumOfStartupFramesPtr</i>	Address where the number of startup frames seen within the last even or odd cycle pair shall be stored.
out	<i>Fr_ConfigParamValuePtr</i>	Address the output value is stored to.

Returns

Std_ReturnType

- E_OK API call finished successfully
- E_NOT_OK API call aborted due to errors

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

